RDSAD: Robust Threat Detection in Evolving Data Streams via Adaptive Latent Dynamics

Abdullah Alsaedi, Zahir Tari, and Redowan Mahmud

***Abstract*—Cyber-Physical Systems (CPSs) are the backbone of Industry 4.0, seamlessly integrating physical and software components for advanced automation in diverse sectors. Recent cyber incidents have shown that these systems are increasingly vulnerable to targeted attacks. Undetected attacks on CPSs can disrupt operations, compromise safety, and cause significant economic losses. Thus, anomaly-based Intrusion Detection Systems (IDSs) are essential to ensure their safety and security, especially in an unsupervised setting where manual labelling is cost-prohibitive. However, current methods encounter significant challenges with complex and evolving characteristics of data streams generated from CPSs, like high dimensionality, uncertainty, and changing patterns, often resulting in high false alarms and missed attacks. This article introduces RDSAD, an unsupervised, robust and adaptive anomaly-based intrusion detection method tailored to effectively monitor evolving complex CPS data streams. RDSAD integrates two innovative components: Dynamic Deviation Recognition (DDR) for capturing the underlying system dynamics, and Shift-aware Model Adaptation (SMA) for adaptive model updates in response to changing patterns. Through extensive evaluations, the experimental results demonstrate the superior performance of RDSAD compared with static and streaming state-of-the-art methods. It achieved the best AUC of 0.90 and 0.88 on SWaT and WADI datasets, respectively, and it obtained efficient runtime with large data streams.**

***Index Terms*—Cyber-physical systems, cybersecurity, anomaly detection, concept drift, deep state-space model.**

I. Introduction

CYBER-PHYSICAL Systems (CPSs) have emerged as essential technologies within Industry 4.0, seamlessly integrating physical and software components. This drives advancements in automation, intelligent decision-making, and connectivity, transforming industrial processes and powering critical infrastructure like smart factories, power plants, and water networks [1]. With the growing prevalence of CPSs, their integration with the Internet has become increasingly widespread. However, this connectivity also exposes their vulnerabilities to various cyber threats. These vulnerabilities, if exploited, can lead to substantial disruptions in their underlying physical processes. Consequently, CPSs face significant risks and threats that can compromise their overall security and integrity [2], [3]. Targeted attacks, involving carefully planned steps to achieve specific goals, have become a major threat [1], [2]. Attackers often manipulate CPS data (i.e., sensor readings, and actuator states) to disrupt operations. Unlike Information Technology (IT) attacks, CPS attacks can have severe consequences, as demonstrated by the targeted attack on the Ukrainian power grid, impacting approximately 225,000 of customers [4]. Similarly, in 2021, hackers attempted a cyberattack on a water treatment plant in Oldsmar, Florida, using a zero-day exploit to gain unauthorised remote access. This enabled them to maliciously alter the chemical levels of the water supply [5]. Several other high-profile incidents, including the Colonial pipeline ransomware attack and Stuxnet-like events, further demonstrate the seriousness of targeted attacks on critical infrastructure [1], [2], [4]. Conventional security measures in IT-based systems (e.g., firewalls and intrusion detection systems) are insufficient for comprehensive CPS protection. Their inability to analyse and detect attack patterns within operational data, such as sensor readings or actuator commands, makes them vulnerable to targeted attacks that manipulate such data at a granular level [2], [6]. These limitations have motivated researchers to develop new intrusion detection methods specifically tailored for CPSs [2], [3], [6], [6], [7]. These methods focus on monitoring CPS data to identify malicious activities like unauthorised changes in sensor readings or actuator commands, potentially indicating attempts to alter the internal operations of the target CPSs [2], [6]. Recently, anomaly-based Intrusion Detection Systems (IDSs) have emerged as a promising solution in CPS security due to their ability to learn from CPS data autonomously, without relying on human intervention [3], [6], [7]. These anomaly-based IDSs can leverage the power of Machine Learning (ML) methods to learn the 'normal' behaviour patterns of the CPS during training. During runtime, they can automatically detect anomalies and deviations from these patterns as potential cyberattacks, enabling preventive actions [1], [2]. While they are effective at detecting such attack patterns, several critical challenges arise.

Challenge 1. Capturing the underlying normal behaviour in complex systems: A core challenge in anomaly detection lies in designing models that effectively learn the underlying normal behaviour "*normality*" in dynamic and unpredictable environments like CPSs [8], [9]. CPSs, as engineered systems, integrate control logic with dynamic environments, leading to

Received 26 February 2024; revised 31 May 2025; accepted 22 September 2025. Date of publication 24 September 2025; date of current version 14 January 2026. *(Corresponding author: Abdullah Alsaedi.)*

Abdullah Alsaedi and Zahir Tari are with the RMIT Centre of Cyber Security Research and Innovation (CCSRI), School of Computing Technologies, RMIT University, Melbourne, VIC 3000, Australia (e-mail: alsaedi.abdullah11@gmail.com; zahir.tari@rmit.edu.au).

Redowan Mahmud is with the School of Electrical Engineering, Computing and Mathematical Sciences, Curtin University, Perth, WA 6102, Australia (e-mail: mdredowan.mahmud@curtin.edu.au).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2025.3614294, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2025.3614294

interactions characterised by uncertainty and complex temporal dependencies. Capturing these intricacies is crucial for accurate detection [8], [9]. Moreover, CPS data often presents additional challenges like high dimensionality and the absence of ground truth labels [10]. Effectively handling these factors while simultaneously accounting for uncertainty and temporal dynamics makes unsupervised anomaly detection in CPSs a significant challenge.

Challenge 2. Adapting to normality shifts in CPSs: CPSs face constant changes due to factors like environmental variations, technology integration, and operational shifts [1]. These changes can lead to *concept drift* [11], where the underlying statistical properties of data evolve over time. In anomaly detection, this manifests as a *normality shift* [12], meaning the definition of the underlying patterns of *normal* behaviour changes. Although significant progress has been made in anomaly-based intrusion detection methods, most existing methods [13], [14], [15], [16], [17] work under a closed-world and static assumption. This means they expect real-time data distribution to relatively match the training data. This assumption might hold true in static environments but hinders their effectiveness in dynamic and evolving system states where data patterns shift unpredictably [11], [18]. For instance, in water treatment plants, variations in raw water quality due to seasonal changes can redefine normal operation [11]. Static-based anomaly detection models, unaware of this evolving normality, might misclassify normal events as anomalies or miss actual attacks, potentially reducing system trust and security [11], [12], [19]. Therefore, deploying adaptive anomaly detection solutions is crucial for CPSs. These solutions can continuously learn and adjust to changing data patterns, ensuring reliable and effective intrusion detection even in dynamic environments [12], [19], [20].

Challenge 3. Overcoming self-poisoning and catastrophic forgetting: Recent works [21], [22], [22], [23], [24], [25] have proposed streaming anomaly detection solutions that continuously update the detection model in real time to address *challenge 2*. However, these solutions often rely on passive updates that encounter additional issues. These updates can be resource-intensive, especially with high-dimensional data streams [11], [26]. More importantly, they face two major challenges: *self-poisoning* [27] and *catastrophic forgetting* [26]. *Self-poisoning* occurs when the model mistakenly learns from attack observations as normal behaviour during the passive updates. This can lead to an inability to accurately detect real attacks in the future [10], [27]. *Catastrophic forgetting* happens when continuous updates cause the model to forget previously learned patterns of normal behaviour as it adapts to new data. This can result in misclassifying legitimate normal events as attacks [26], [27]. The combined impact of these challenges can be significant, resulting in increased false alarms, missed attacks, and reduced system trust. Addressing them is crucial for developing robust and reliable threat detection methods capable of adapting to evolving normal behaviour while preserving their knowledge base.

To address the aforementioned limitations, we introduce a novel *unsupervised* anomaly-based threat detection method, called **R**obust Attack **D**etection for Evolving Data **S**treams via **A**daptive Latent **D**ynamics Learning (*RDSAD*). It is specifically tailored for real-time monitoring of complex CPS data streams. It effectively captures the underlying normal behaviour, detects deviations as attack patterns, and adaptively updates its detection model to accommodate changing behaviours. It achieves this through two key components: *Dynamic Deviation Recognition* (DDR) and *Shift-aware Model Adaptation* (SMA).

To address the *first challenge*, the DDR integrates a novel neural architecture, inspired by deep state space-based generative models [8], [9], to effectively capture low-dimensional latent dynamics of the underlying normal behaviour of monitored systems. Unlike prior works [13], [14], [15], [21], [22], [23], [24], [25], DDR explicitly considers both the uncertainty and temporal dependencies in data streams through two distinct latent states: *continuous states* for temporal learning and *stochastic states* for uncertainty modelling, leading to improved inference and more effective threat detection. Additionally, DDR incorporates a statistics-driven emission approach [18], [28], with density reconstruction and moving averages, further enhancing robustness in dynamic environments.

To effectively adapt to the evolving environments and normality shifts (*the second challenge*), the SMA continuously monitors for shifts in the underlying latent dynamics and adaptively updates the model in response. Contrary to existing methods [21], [22], [23], [24], [25] that are prone to catastrophic forgetting due to continuous adaptation and self-poisoning from misinterpreting attack or anomalous observations (*the third challenge*), SMA employs a *robust update decision* approach to prevent self-poisoning and an *adaptive regularisation term* to preserve previously acquired knowledge, ensuring model integrity and reliability. The key contributions of this paper are outlined as follows:

- Developed a novel unsupervised anomaly-based threat detection method, RDSAD, that learns and adapts to the evolving normal behaviours of the monitored system, enabling robust and accurate detection of potential attacks.
- Designed a *Dynamic Deviation Recognition* (DDR) based on an innovative neural architecture, enabling robust representation of system dynamics and effective threat detection.
- Introduced a *Shift-aware Model Adaptation* (DMA) method that continually tracks shifts in underlying dynamics. It adaptively updates RDSAD in response to these shifts through a robust update decision. This minimises the risk of self-poisoning from anomalous observations, ensuring sustained model accuracy and reliability. Additionally, it preserves the integrity of accumulated knowledge via an adaptive regularisation term.
- Conducted comprehensive experiments and evaluations on four distinct datasets to evaluate the performance of the proposed RDSAD. The experimental results demonstrate that RDSAD outperforms state-of-the-art methods in terms of both performance metrics and runtime efficiency.

The paper is structured as follows: Section II introduces some basic notations and concepts. Section III details the proposed RDSAD. Section IV presents the related works. Section V outlines the experimental setup, followed by Section VI, which

TABLE I
DESCRIPTION OF NOTATIONS

Symbols	Description
$\boldsymbol{x}$	the observation vector
t	timestamp
T	the window length
ϕ	the parameters of inference model
θ	the parameters of emission model
$\boldsymbol{z}$	the stochastic latent state
$\boldsymbol{h}$	the continuous latent state
$\mathcal{F}_h(.)$	the flow function
$\mathcal{ST}$	the Student-t distribution
ℓ	the likelihood value
s	deviation score
η	deviation threshold
δ	change score
Δ	the shift intensity
Υ	shift threshold
D_{KL}	*Kullback-Leibler* Divergence

discusses the experimental results. Finally, Section VII provides a summary and concludes the paper.

II. PRELIMINARIES AND BACKGROUND

A. Basic Notation

This paper presents vectors as bold lower-case letters, such as $\boldsymbol{x}$. Matrices are denoted in bold upper-case letters, such as $\boldsymbol{X}$. The time index is denoted as subscript x_t. A sequence of observations from time 1 to t is represented as $\boldsymbol{x}_{1:t}$. The necessary notations for the proposed solution are outlined in Table I.

B. State-Space Models

This section briefly introduces State-Space Models (SSMs) since RDSAD works on an SSM-like model. SSMs are widely used to model sequential data by learning the relationship between latent variable $\boldsymbol{z}_t$ and observed variable $\boldsymbol{x}_t$. It is assumed that an observation $\boldsymbol{x}_t$ depends directly on a latent process $\boldsymbol{z}_t$:

$$\boldsymbol{z}_t = f_\theta(\boldsymbol{z}_{t-1}, \boldsymbol{x}_t) + \epsilon_t, \qquad \text{(latent dynamics)} \tag{1a}$$

$$\bar{\boldsymbol{x}}_t = g_\theta(\boldsymbol{z}_t). \qquad \text{(emission)} \tag{1b}$$

where $\boldsymbol{z} \in \mathbb{R}^L$, $\boldsymbol{x} \in \mathbb{R}^m$, and θ represents the model parameters. f_θ is the transition function that governs the latent dynamics (i.e., it defines the evolution of the state over time), while g_θ is the emission function, which outlines the relationship between the latent state and the observed data. SSMs offer an elegant framework for modelling and estimating densities of data streams. Nonetheless, classical SSMs are often limited to linear models, thereby lacking the capacity to effectively model complex real-world data streams.

C. Deep SSMs and Variational Inference

The State-Space Models (SSMs) can be effectively extended by incorporating Neural Networks (NNs) into the structure, leading to Deep State-Space Models (Deep SSMs) [8], [9]. These deep SSMs can capture the characteristics of latent dynamics $\boldsymbol{z}$, achieved through the computation of the posterior distribution $p_\theta(\boldsymbol{z}|\boldsymbol{x}) = \frac{p_\theta(\boldsymbol{x}_t|\boldsymbol{z})p_\theta(\boldsymbol{z})}{p_\theta(\boldsymbol{x})}$, where $p_\theta(\boldsymbol{z})$ denotes the prior distribution over the latent dynamics and θ is the model parameter. However, direct computation of $p_\theta(\boldsymbol{z}|\boldsymbol{x})$ is a challenging task except for a simple model, mainly due to the intractable integral involved in computing the marginal likelihood $p_\theta(\boldsymbol{x}) = \int p_\theta(\boldsymbol{x}|\boldsymbol{z})\ p_\theta(\boldsymbol{z})d\boldsymbol{z}$ [29]. As a solution to this issue, variational inference [30] is applied to estimate an approximate distribution of the true posterior, denoted as $q_\phi(\boldsymbol{z}|\boldsymbol{x})$. Therefore, the problem is formulated as an optimization problem to maximize the Evidence Lower Bound (ELBO), $\mathcal{L}$ formulated as:

$$\mathcal{L}(\phi, \theta; \boldsymbol{x}) = \mathbb{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})}[\log p_\theta(\boldsymbol{x}|\boldsymbol{z})] - D_{KL}(q_\phi(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z})) \tag{2}$$

where the Kullback-Leibler divergence, D_{KL}, ensures that the approximate distribution $q(\boldsymbol{z}|\boldsymbol{x})$ is close to the prior distribution $p(\boldsymbol{z})$. The $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ and $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ are the inference and generative models, respectively. These models are constructed using deep neural networks and then trained jointly via backpropagation, leveraging the reparameterisation trick for gradient computation [30]. Lastly, Recurrent Neural Networks (RNNs) can be employed to establish connections between the SSMs and Variational Autoencoders (VAEs) [8], enabling the acquisition of non-linear State-Space Models (SSMs) that incorporate the latent dynamics and emission models as outlined in (1a) and (1b), thereby offering a powerful approach to handle complex, real-world sequential data streams.

D. Neural Ordinary Differential Equations

Neural Ordinary differential equations (neural ODEs) are a significant advancement in modelling dynamical systems, given their capability to capture long-term dependencies and manage irregularly sampled time series data [31]. Traditional approaches to modelling time-series and sequential data, such as Recurrent Neural Networks (RNNs), involve applying a sequence of transformations to a hidden state at fixed intervals:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t) \tag{3}$$

where $t \in \{0, \ldots, T\}$ and $\boldsymbol{h}$ is the hidden state. In contrast, neural ODEs describe the continuous evolution of hidden states using an ordinary differential equation: $\boldsymbol{h} = f(\boldsymbol{h}(t), t)$, where f acts as a neural network. Given an initial value $h(t_0)$, this dynamic can yield results at any time point t_1:

$$\boldsymbol{h}(t_1) = \boldsymbol{h}(t_0) + \int_{t_0}^{t_1} f(\boldsymbol{h}(t), t)dt = \text{ODESolve}(\boldsymbol{h}(t_0), f, t_0, t_1). \tag{4}$$

However, the integral in (4) requires expensive numerical solvers, resulting in high computation costs. To overcome this issue, *Neural Flow* [31] was proposed as a recent advancement in neural ODEs. It models the solution curve of ODE directly with a neural network, substantially lowering computation costs. Neural ODEs and *Neural Flow* have proven practical and effective approaches in modelling dynamic systems handling irregularly-sampled data streams.

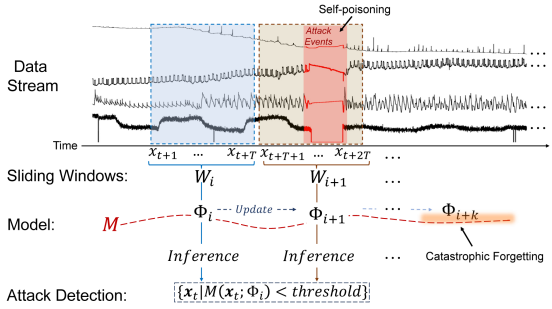


Fig. 1. Adaptive anomaly-based intrusion detection in evolving data streams.

E. Problem Definition

Let $\mathcal{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \dots\}$ represent high-dimensional observations from CPSs arriving in a continuous stream. Given the unbounded nature of such data streams, we employ sliding windows of size T to process the data in real-time. A sliding window W, starting at time step t, is defined as a sequence of T consecutive observations, $\mathbf{x}_{t+1:t+T} = \{\mathbf{x}_{t+1}, \dots, \mathbf{x}_{t+T}\}$, where each $\boldsymbol{x}_t \in \mathbb{R}^m$ is an m-dimensional vector associated with a timestamp t.

Adaptive unsupervised anomaly-based intrusion detection aims to identify unusual patterns in these data streams that deviate from established normal behaviour, potentially indicating attacks and system anomalies. As depicted in Fig. 1, the detection model $\mathcal{M}$, parameterised by Φ, continuously analyses data through sliding window W of size T. The model, $\mathcal{M}$, performs inference on this window, generating a set of anomaly scores for the observations within W, denoted as $\{\mathcal{M}(x_i; \Phi) | x_i \in W\}$. Considering probability-based scoring, observations with scores below a defined threshold, suggesting they have a low probability of being normal, are flagged as potential attacks. The model $\mathcal{M}$ then updates its parameters Φ in an unsupervised manner, incorporating the latest data from the stream to refine its detection capabilities.

A key challenge in CPS anomaly detection lies in effectively capturing the underlying patterns of normal behaviour. This task is compounded by the inherent uncertainty and temporal dependencies within the high-dimensional data streams generated by CPSs [11]. In this context, each variable in data streams is perceived as a stochastic process produced with auto-correlated temporal properties [8], [10]. Furthermore, the dynamic nature of CPSs often leads to changes in the underlying patterns of the normal behaviour, known as *normality shift* [11], [12]. This poses a critical challenge to maintaining accurate anomaly detection over time.

Definition 1 (Normality Shift [11], [12]): Given a time period $[0, t]$, and a window of observations $\boldsymbol{x}_{0:t}=\{\boldsymbol{x}_0, \boldsymbol{x}_1, \dots, \boldsymbol{x}_t\}$, following a certain distribution $P_{0:t}(\boldsymbol{x})$. Normality shift occurs at time $t+1$ if $\exists t : P_t(\boldsymbol{x}) \neq P_{t+1}(\boldsymbol{x})$, indicating a covariate shift in the distribution of input data streams.

Building upon the challenges of adapting to normality shifts, continuous learning through passive updates encounters additional issues: *self-poisoning* [27] and *catastrophic forgetting* [26]. We formally define self-poisoning (highlighted in Fig. 1 by the red shaded area) as:

Definition 2 (Self-Poisoning [27], [32]): Let $\mathcal{A}$ represent a set of attack observations, self-poisoning occurs when a detection model $\mathcal{M}$ with parameters Φ inadvertently learns from $\mathcal{A}$ during its continual update process, compromising its ability to accurately detect future attacks.

Catastrophic forgetting (depicted as the trailing-off pattern in Fig. 1) is formally defined as:

Definition 3 (Catastrophic Forgetting [26]): Let $\mathcal{H}$ represent a set of historical observations reflecting normal behaviour. Catastrophic forgetting occurs when the performance of $\mathcal{M}$ on $\mathcal{H}$ decreases significantly after being continuously updated with new data. This happens because new information gradually replaces information about past normal behaviour within the model's internal parameters Φ.

III. The Proposed Solution

This section discusses RDSAD in more detail. Fundamentally, RDSAD adaptively learns from the evolving normal behaviours of the monitored system, ensuring robust and accurate detection of attacks even as the system changes. The effectiveness of RDSAD stems from its two essential components: *Dynamic Deviation Recognition (DDR)* and *Shift-aware Model Adaptation (SMA)* as depicted in Fig. 2. The DDR component is dedicated to learning and capturing the underlying latent dynamics of the monitored system, with any significant deviations from these learned dynamics detected as potential threats. It accomplishes this through its two integral phases: *System Representation Learning (SRL)* and *Deviation Detection (DD)*, which together form an effective and robust threat detection method. The SMA enhances RDSAD with robust and adaptive model updates in real time. This adaptability is achieved through two operations: *latent-space shift identification* and *robust update decision*. In the following sections, a detailed discussion of each component is provided, beginning with the two phases of DDR followed by SMA.

A. System Representation Learning (SRL)

SRL aims to learn the underlying latent dynamics that govern the normal behaviour of monitored systems, transforming high-dimensional observations into effective low-dimensional latent representations. While real-world CPS data is often high-dimensional, many physical systems operate on low-dimensional latent dynamics [2], [3], [7]. These underlying dynamics can be effectively captured by a few latent state variables that represent the principal properties of the monitored systems. Due to their reduced dimensionality, these representations are computationally efficient and memory-friendly, making them optimal for monitoring system operations. The SRL comprises a Temporal Embedding Network (TEN) and an inference model. TEN serves as a feature extraction layer that processes input data streams, converting them into a form suitable for the inference model. Its primary function is to enhance the model's capability to learn from the data streams, thereby enabling an effective representation of the system's underlying dynamics. The inference model, working in conjunction with TEN, leverages the

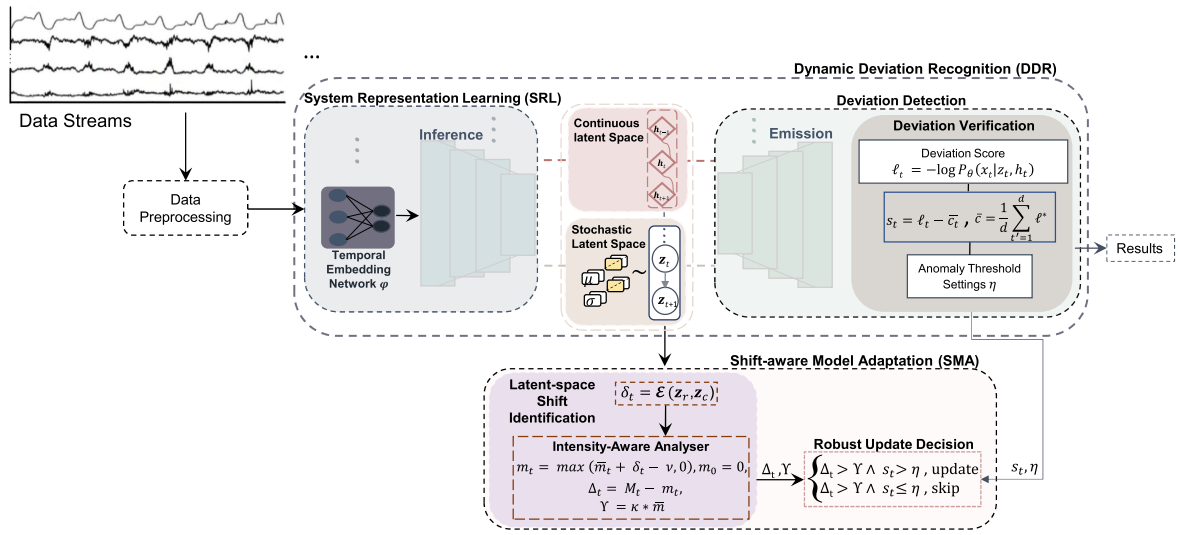


Fig. 2. The overview of RDSAD.

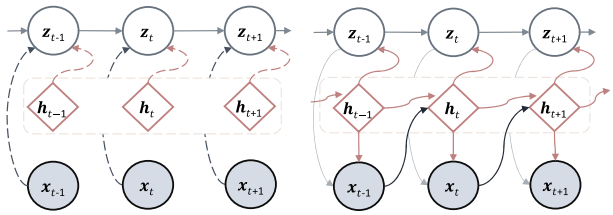
temporally extracted features to model the probability distribution $p(\boldsymbol{z_t}|\boldsymbol{x_{1:t}})$, where $\boldsymbol{z_t}$ corresponds to compact, time-variant latent states representing the system's normal behaviour over time. In what follows, we discuss the TEN and the inference model in more detail.

1) Temporal Embedding Network: Temporal Embedding Network (TEN), denoted as φ, is constructed using Temporal Convolutional Networks (TCN) [33], which are known to be more time-efficient than traditional RNN-based structures, and can often produce superior prediction performance with sequential data. TCN is based on a 1-D convolution network that operates on temporal dimensions using a dilation factor d. Formally, it is defined as:

$$\varphi(\boldsymbol{x}) = \sum_{i=0}^{k-1} f(i) \cdot \boldsymbol{x}_{s-d\cdot i} \tag{5}$$

where $f \in \mathbb{R}^{k\times d}$ refers to the convolutional filter of size k. $s - d \cdot i$ indicates the direction of past information in the streams. d corresponds to the dilation factor. When $d = 1$, this forms a regular convolution where the filter is applied to every sequential element in the series. However, as d increases, the filter is applied at larger gaps in the input series. This expands the receptive field of the network to capture long-term patterns in the data.

2) Inference Model: The inference model plays a crucial role within SRL by capturing the latent dynamics of the observed data streams. Specifically, it models the posterior distributions of the input observations $\boldsymbol{x}$ as compact latent dynamics (e.g., $\boldsymbol{z}$ and $\boldsymbol{h}$), represented as $p_\theta(\boldsymbol{z}, \boldsymbol{h}|\boldsymbol{x})$. However, this direct modelling of the posterior becomes intractable due to the inherently complex integration involved (as discussed in Section II-C). Consequently, we leverage a filtering-based structured variational inference [29]. This works particularly well for streaming data by approximating the posterior distribution of the current latent dynamics based only on past and current information. This approach creates a structured approximate posterior with a



(a) Inference Model. (b) Emission Model.

Fig. 3. Graphical demonstration of inference and emission models. Nodes correspond to various variables, and edges indicate the dependencies between them. Circular nodes represent latent stochastic variables, and diamond-shaped nodes correspond to GRU-flow variables.

factorised q_ϕ distribution as follows:

$$q_\phi(\boldsymbol{z}, \boldsymbol{h}|\boldsymbol{x}) = \prod_{t=1}^{T} q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}, \boldsymbol{h}_t, \boldsymbol{x}_t), \tag{6a}$$

$$q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}, \boldsymbol{h}_t, \boldsymbol{x}_t) \sim \mathcal{N}\left(\boldsymbol{\mu}_{z_t}, diag(\boldsymbol{\sigma}^2_{z_t})\right), \tag{6b}$$

$$\boldsymbol{\mu}_{z_t} = \zeta^\mu_\phi(\boldsymbol{z}_{t-1}, \boldsymbol{h}_t, \varphi(\boldsymbol{x}_t)), \tag{6c}$$

$$\boldsymbol{\sigma}_{z_t} = \text{Softplus}[\zeta^\sigma_\phi(\boldsymbol{z}_{t-1}, \boldsymbol{h}_t, \varphi(\boldsymbol{x}_t))]. \tag{6d}$$

The approximate posterior is modelled as a diagonal multivariate Gaussian distribution, where $\boldsymbol{\mu}_{z_t}$ and $\boldsymbol{\sigma}_{z_t}$ represent the mean and variance, respectively. These parameters are determined by two fully-connected neural networks with a single hidden layer, denoted as ζ^μ_ϕ and ζ^σ_ϕ, respectively. The softplus activation function, $\text{Softplus}(x) = \log(1 + e^x)$, is utilised to ensure a positive and nonlinear transition from the latent states to σ. Finally, ϕ refers to the parameter sets of the inference model, while φ denotes the Temporal Embedding Network (TEN) (discussed in Section III-A1). A graph illustration of the inference model is depicted in Fig. 3(a) in which the current latent variable $\boldsymbol{z}_t$ conditioned on the previous latent variable

$\boldsymbol{z}_{t-1}$, the current hidden state $\boldsymbol{h}_t$, and the current observation $\boldsymbol{x}_t$. In RDSAD, the internal dynamics of the underlying behaviours are captured through the interaction of two distinct latent states: (1) *Continuous Latent States* ($\boldsymbol{h}$) and (2) *Stochastic Latent States* ($\boldsymbol{z}$). Together, these latent states provide a robust and compact representation, thereby enhancing the ability of RDSAD to detect potential deviations in the monitored system.

Continuous latent states: They capture the long-term temporal dependencies and transitions of the latent variables by tracking their continuous evolution. This allows the model to effectively account for temporal dependencies within the data. We model the transition function of $\boldsymbol{h}$ with ideas from neural Ordinary Differential Equations (neural ODEs). In particular, we utilise GRU-flow [31] as the transition function to model the continuous latent state $\boldsymbol{h}_i \in \mathbb{R}^h$. GRU-flow models the continuous evolution of latent states $\boldsymbol{h}$ and captures their temporal dependencies in a flexible and adaptive manner using an ordinary differential equation. As detailed in Section II-D, this approach leverages the strengths of neural ODEs and *Neural Flow* [31], providing an effective way to capture the continuous dynamics of the system. The evolution of the latent state in continuous time is defined as:

$$\mathbf{u}(t,\boldsymbol{h}) = \alpha_u \cdot \text{sigmoid}(f_{\mathbf{u}}(t,\boldsymbol{h},\varphi(\boldsymbol{x}))) \tag{7a}$$

$$\mathbf{r}(t,\boldsymbol{h}) = \alpha_r \cdot \text{sigmoid}(f_{\mathbf{r}}(t,\boldsymbol{h},\varphi(\boldsymbol{x}))) \tag{7b}$$

$$\tilde{\mathbf{h}}(t,\boldsymbol{h}) = \tanh(f_{\tilde{\mathbf{h}}}(t,\mathbf{r}(t,\boldsymbol{h}) \odot \boldsymbol{h},\varphi(\boldsymbol{x}))) \tag{7c}$$

$$\mathcal{F}_h(t,\boldsymbol{h}) = \boldsymbol{h} + g(t)\,(1-\mathbf{u}(t,\boldsymbol{h})) \odot \left(\tilde{\mathbf{h}}(t,\boldsymbol{h}) - \boldsymbol{h}\right) \tag{7d}$$

where $\mathbf{u}(.)$ and $\mathbf{r}(.)$ denote the update and reset gates, respectively. The parameters α_u and α_r are scalars $\in \mathbb{R}$ that control the impact of the gates and maintain the property of invertibility in the flow, thereby facilitating the learning of long-term dependencies and providing greater control over the state evolution. $\tilde{\mathbf{h}}(.)$ represents the intermediate hidden state. The functions $f_{\mathbf{u}}, f_{\mathbf{r}}$ and $f_{\tilde{\mathbf{h}}}$ are fully-connected neural networks that represent the transformations applied to compute the gates and the candidate hidden state. g is a continuous function that satisfies $g(0) = 0$, and $\odot$ denotes element-wise multiplication. $\mathcal{F}_h(.)$ denotes the flow function which models the evolution of the GRU state $\boldsymbol{h}$ in continuous time.

Stochastic latent states: These state variables capture latent dynamics generating the data and their associated uncertainty. Each stochastic latent variable $\boldsymbol{z}_t$ is conditioned on both the previous stochastic variable $\boldsymbol{z}_{t-1}$ and the current continuous state $\boldsymbol{h}_t$. This allows it to extract information about the historical context (from $\boldsymbol{z}_{t-1}$) and the current system context (from $\boldsymbol{h}_t$), making it more reflective of the actual evolving state of the system. This richer representation allows us to track subtle changes in the latent dynamics, which can manifest as normality shifts in the observed data (see Section III-D1). We employ a transition distribution for $\boldsymbol{z}$ that is characterised by a multivariate Gaussian distribution, defined as follows:

$$p_\theta(\boldsymbol{z}_t|\boldsymbol{z}_{t-1},\boldsymbol{h}_t) \sim \mathcal{N}\left(\hat{\boldsymbol{\mu}}_{z_t}, diag(\hat{\boldsymbol{\sigma}}^2_{z_t})\right) \tag{8a}$$

$$\hat{\boldsymbol{\mu}}_{z_t} = \zeta^{\hat{\mu}}_\theta(\boldsymbol{z}_{t-1},\boldsymbol{h}_t), \tag{8b}$$

$$\hat{\boldsymbol{\sigma}}_{z_t} = \text{Softplus}[\zeta^{\hat{\sigma}}_\theta(\boldsymbol{z}_{t-1},\boldsymbol{h}_t)]. \tag{8c}$$

$\hat{\boldsymbol{\mu}}_{z_t}$ and $\hat{\boldsymbol{\sigma}}_{z_t}$ are the mean and variance of the transition distribution, respectively. $\hat{\boldsymbol{\mu}}_{z_t}$ is derived from a fully-connected neural network $\zeta^{\hat{\mu}}_\theta$ with a ReLU function, and $\hat{\boldsymbol{\sigma}}_{z_t}$ generated by $\zeta^{\hat{\sigma}}_\theta$ with the Softplus activation function. By considering the uncertainty, the stochastic latent states can incorporate a certain level of randomness or variability, which can potentially improve the capability of the model to handle complex and noisy data streams.

B. Deviation Detection

RDSAD utilises a dynamic deviation detection approach that combines density reconstruction estimation with moving averages. This method assigns deviation scores to incoming observations for real-time intrusion detection. Under the density-based approach, normal observations typically exhibit normal behaviours and have higher reconstruction densities. Conversely, anomalies deviate from this normality and are statistically rare, leading to lower densities [34]. RDSAD leverages this principle based on two main components: emission model and deviation verification.

1) Emission Model: The emission model is constructed with the ability to map the latent dynamics to sufficient statistics of certain conditional distributions of observations $p_\theta(\boldsymbol{x_t}|\boldsymbol{z}_t,\boldsymbol{h}_t)$, which relies on a sample from latent dynamics $\boldsymbol{z}_t$ and $\boldsymbol{h}_t$ as shown in Fig. 3(b). Current works in generative time-series models have commonly employed Gaussian distributions as an emission model [8], [9]. However, Gaussian distributions are known to be sensitive to extreme values and fluctuations, a common characteristic of anomalies. This can lead the emission model to inadvertently fit both normal and anomaly data simultaneously. This simultaneous fitting leads to an inability to accurately identify true anomalies, compromising the effectiveness of the detection system [18]. To address this, we adopt the Student-t distribution, a heavy-tailed distribution that is based on the principles of robust statistics [18], [28]. The Student-t distribution is more robust to extreme values and fluctuations, allowing for more accurate identification of true anomalies compared to a Gaussian assumption. Formally, we employ a Student-t distribution with a location-scale generalisation as the emission model as follows:

$$p(x \mid v,\mu,\sigma) = \frac{\boldsymbol{\gamma}\left(\frac{v+1}{2}\right)}{\boldsymbol{\gamma}\left(\frac{v}{2}\right)\sqrt{v\pi}\sigma}\left[1+\frac{1}{v}\left(\frac{x-\mu}{\sigma}\right)^2\right]^{-\frac{v+1}{2}}, \tag{9}$$

where $\boldsymbol{\gamma}$ denotes the gamma function and v represents the degrees of freedom. In this context, the emission model based on sufficient statistics can be expressed as:

$$p_\theta(\boldsymbol{x_t}|\boldsymbol{z}_t,\boldsymbol{h}_t) = \mathcal{ST}\left(\boldsymbol{x}_t; v_{x_t}, \mu_{x_t}, \sigma_{x_t}\right). \tag{10}$$

where θ represents the parameter sets of the emission model, and $\mathcal{ST}$ denotes the Student-t distribution. v_{x_t}, μ_{x_t}, and σ_{x_t} correspond to the sufficient statistics of the emission distribution. These statistics are parameterised by a neural network as

follows:

$$v_{x_t} = \text{Softplus}[\zeta_\theta^v(\boldsymbol{z}_t, \boldsymbol{h}_t)] + 1 \tag{11a}$$

$$\mu_{x_t} = \zeta_\theta^\mu(\boldsymbol{z}_t, \boldsymbol{h}_t) \tag{11b}$$

$$\sigma_{x_t} = \text{Softplus}[\zeta_\theta^\sigma(\boldsymbol{z}_t, \boldsymbol{h}_t)] \tag{11c}$$

where the functions ζ_θ^v, ζ_θ^μ and ζ_θ^σ are fully-connected neural networks with one hidden layer. The heaviness of the tail is managed by the degree of freedom parameter v. Unlike traditional approaches that consider v as a time-invariant constant, in our configuration, v is parameterised using a neural network to allow it to be data-driven and adaptable over time [28]. By adopting the Student-t distribution and parameterising its components dynamically, our emission model is robust to extreme values and fluctuations, leading to more accurate intrusion detection. Compared to other heavy-tailed distributions (e.g., Cauchy and Laplace), the Student-t distribution offers a more stable and adaptable formulation due to its tunable degrees of freedom, which balance tail heaviness and variance control [18], [28].

2) Deviation Verification: The deviation verification involves two primary steps: calculating a *deviation score* and establishing a *deviation threshold*.

Deviation Score: Building upon the emission model and density reconstruction, we calculate a deviation score to quantify how much an observation deviates from normal behaviour. We leverage the log-likelihood of the observation $\boldsymbol{x}_t$ conditioned on latent dynamics $\boldsymbol{z}_t$ and $\boldsymbol{h}_t$ as follows:

$$\ell_t = \log p_\theta(\boldsymbol{x}_t | \boldsymbol{z}_t, \boldsymbol{h}_t). \tag{12}$$

where ℓ_t denotes likelihood value. High log-likelihood indicates good reconstruction, aligning with normal behaviour, while low values suggest deviations (i.e., anomalies). This approach allows for real-time requirements and efficient processing in streaming settings [8]. To quantify this deviation, we define the deviation score s_t at time t as the difference between the current likelihood and a measure of "consistent normal behaviour" $\bar{c}_t$:

$$s_t = \ell_t - \bar{c}_t, \tag{13a}$$

$$\bar{c}_t = \frac{1}{d} \sum_{t'=1}^{d} \ell^*. \tag{13b}$$

Here, $\bar{c}_t$ dynamically captures this normal behaviour through a moving average of past likelihood scores ℓ^* for normal observations within a window of size d. This is more robust and adaptable than raw log-likelihood scores, as it considers the recent context of normal behaviour. For instance, transient changes or noise would not drastically affect the average compared to a single score, providing a more accurate intrusion detection in non-stationary environments. Next, We describe how to automatically establish the deviation threshold, which utilises the deviation score to drive the final detection decision.

Deviation Threshold Settings: The deviation threshold serves as the criterion for determining whether an observation significantly deviates from normal behaviour. We automatically set this threshold using the principle of Extreme Value Theory (EVT) [18], a statistical approach for analysing extreme values in data distribution. EVT, specifically the Peaks-Over-Threshold (POT) theorem [18], helps us focus on the tail of the deviation score distribution, where potential anomalies are more likely to reside. We leverage a Generalized Pareto distribution (GPD) to model this tail, and the threshold is obtained by fitting the tail proportion of a GPD as follows:

$$\bar{F}(s) = P(\varepsilon - \boldsymbol{s} > s | \boldsymbol{s} < \varepsilon) \sim \left(1 + \frac{\xi s}{\beta}\right)^{-\frac{1}{\xi}}. \tag{14}$$

where $\bar{F}$ represents the tail of the distribution, and ε is the initial threshold of deviation scores. The parameters ξ and β correspond to the shape and scale of GPD, respectively. They are estimated through The Maximum Likelihood Estimation (MLE). The term $\varepsilon - \boldsymbol{s}$ refers to the portion below the initial threshold ε, which is empirically set to a low quantile. The final threshold η is obtained as follows:

$$\eta \simeq \varepsilon - \frac{\hat{\beta}}{\hat{\xi}} \left(\left(\frac{N \cdot \rho}{N_\varepsilon} \right)^{-\hat{\xi}} - 1 \right). \tag{15}$$

where ρ is the target probability level for observing the event $\boldsymbol{s} < \varepsilon$, N is the number of observations, and N_ε is the number of s_i that are less than the threshold value ε (i.e., the number of s_i s.t. $s_i < \varepsilon$). Two parameters involved in POT require tuning: ρ and low quantile. They can be established empirically (see Section V-D). These model-wide parameters allow us to adaptively adjust the deviation threshold to accommodate potential changes in the data over time. This adaptive nature is further enhanced by the Shift-aware Model Adaptation (SMA) process (Section III-D), which recalculates the threshold based on the latest data when changes are detected.

Algorithm 1: Deviation Threshold.

1: **procedure** DEVIATIONTHRESHOLD($\boldsymbol{s}$,q)
2: Initialise ε from $(s_1, \ldots, s_n)$ such that ε is a low quantile.
3: Construct $Y = \{\varepsilon - s_i | s_i < \varepsilon, \forall i \in [1, n]\}$
4: Estimate parameters $\hat{\xi}$ and $\hat{\beta}$ using MLE(Y)
5: Compute the final threshold η using Eq. 15
6: **return** η
7: **end procedure**

Algorithm 1 summarises the initialisation steps for the deviation threshold. The initial threshold ε is set as the low quantile of the deviation score samples $(s_1, \ldots, s_n)$, where n is the number of samples in $\boldsymbol{s}$. In Line 3, we construct the set Y (i.e., "a portion below the threshold"), which includes the differences between the initial threshold ε and each score s_i that is below ε. This set is then used to fit a GPD, enabling us to infer the distribution of the lower extreme values. The final threshold η is computed according to (15).

C. Model Training

This section details the training process of RDSAD, which jointly optimises the inference parameters (ϕ) and emission parameters (θ) to learn effective latent representations within the "latent dynamic space". This space captures the underlying structure and dynamics of the data streams, enabling accurate deviation detection and adaptation. We achieve this joint optimisation through Stochastic Gradient Variational Bayes (SGVB)

estimator [30], which maximises the *Evidence Lower BOund* (ELBO) objective $\mathcal{L}$:

$$\mathcal{L}(\phi,\theta)=\sum_{t=1}^{T}\Bigg[\mathbb{E}_{q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1},\boldsymbol{h}_t,\boldsymbol{x}_t)}\Bigg[\underbrace{\log p_\theta(\boldsymbol{x_t}|\boldsymbol{z}_t,\boldsymbol{h}_t)}_{\text{reconstruction term}}\Bigg] - \underbrace{D_{KL}\left(q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1},\boldsymbol{h}_t,\boldsymbol{x}_t)\parallel p_\theta(\boldsymbol{z}_t|\boldsymbol{z}_{t-1},\boldsymbol{h}_t)\right)}_{\text{KL-divergence term}}\Bigg] \tag{16}$$

The ELBO comprises two main terms. The first term is the 'reconstruction term', which represents the log-likelihood of the data streams. It ensures the ability of the model to accurately reconstruct the input data $\boldsymbol{x}_t$ from the latent dynamics $\boldsymbol{z}_t$ and $\boldsymbol{h}_t$. The second term, the 'KL-divergence term', guides the inference model towards learning informative latent dynamics. This KL-divergence term measures the difference between the learned variational distribution (q_ϕ) and a prior distribution (p_θ) on the latent dynamics, ensuring they capture meaningful information. The expectation $\mathbb{E}$ is approximated using Monte Carlo integration [29]. Instead of directly sampling $\boldsymbol{z}_t$ from q_ϕ, we use a reparameterisation trick [30], where we sample an auxiliary random variable $\boldsymbol{\epsilon}$ from a standard normal distribution $\mathcal{N}(0,\boldsymbol{I})$ and then obtain $\boldsymbol{z}_t$ through a differentiable transformation $g_\phi(.)$ as $\boldsymbol{z}_t=g_\phi(\boldsymbol{x}_t,\boldsymbol{\epsilon})$. This allows us to obtain a differentiable estimator of the evidence lower bound $\mathcal{L}$ in (16) during optimisation, as it permits gradients to flow back through the network.

D. Shift-Aware Model Adaptation (SMA)

This section presents the Shift-aware Model Adaptation (SMA). Unlike conventional approaches that necessitate frequent model updates [13], [14], [15], [16], [17], RDSAD incorporates SMA, which provides robust and adaptive learning capabilities in real time. SMA continually monitors changes in the underlying system dynamics, triggering an update only when significant changes are identified. By selectively updating the model based on relevant changes, it ensures that the model maintains accurate and effective in capturing the evolving dynamics of the system. This is to minimise unnecessary updates, which can be computationally expensive and potentially fail to capture the historical semantic patterns. SMA involves two key steps: 1) *Latent-space shift identification* and 2) *robust update decision*. In the subsequent sections, we discuss each step.

1) Latent-Space Shift Identification: The goal here is to track and identify normality shifts in system dynamics within the stochastic latent space. To achieve this, we introduce two latent windows of equal length T named the *reference* and *current* window. The *reference* window, represented as $\boldsymbol{z}_r=\{\boldsymbol{z}_{t-T},\ldots,\boldsymbol{z}_{t-1}\}$, captures the historical samples and serves as a baseline for comparison, remaining fixed unless a shift is detected. The *current* window, denoted as $\boldsymbol{z}_c=\{\boldsymbol{z}_{t-T+1},\ldots,\boldsymbol{z}_t\}$, continuously updates with the latest stochastic latent variables, reflecting the evolving characteristics of the latent dynamics.

To quantify the difference between these windows and detect potential shifts, we adopt a variant of the energy distance metric [35]. This metric has several desirable statistical properties that enable us to quantitatively measure changes between two sets of samples. By computing the energy distance, we derive a change score δ. Formally, the change score δ is defined by the energy distance $\mathcal{E}(.,.)$, which is computed as the weighted average of squared distances between two sets of samples in z-space as follows:

$$\delta_t=\mathcal{E}(\boldsymbol{z}_r,\boldsymbol{z}_c):=\frac{2}{T(T-1)}\sum_{i=1}^{T-1}\sum_{j=i+1}^{T}\left\|z_r^{(i)}-z_c^{(j)}\right\|^2. \tag{17}$$

where $\|.\|$ denotes the euclidean norm. Typically, δ approaches a value close to 0 when the two sets of samples, $\boldsymbol{z}_r$ and $\boldsymbol{z}_c$, are drawn from similar distributions, indicating a high degree of similarity. Conversely, δ increases as the difference between the two sample sets grows. The change score δ (17) provides a quantitative measure of changes in the latent space, allowing us to track shifts in the system's dynamics over time. However, to distinguish between transient fluctuations and significant changes in the underlying system dynamics, we need to effectively quantify the intensity and significance of the detected changes. To this end, we leverage the intensity-aware analyser [36], which builds upon the computed change scores δ. This analyser unitises concepts from a CUSUM-like test, allowing us to track the presence of changes and their intensity and persistence over time. The intensity of the changes is measured through the cumulative variable m_t, defined in (18), which maintains the accumulated difference between the recently observed score values and the average of previously observed values.

$$m_t=\max(\bar{m}_t+\delta_t-\nu,0), m_0=0. \tag{18}$$

Here, $\bar{m}_t$ is the calculated mean of the observed change score δ_t at time t, and ν is an adjustable parameter which represents the acceptable level of change, typically assigned a value near zero [36]. Finally, the intensity Δ_t at time t defines as $\Delta_t=M_t-m_t$, where $M_t=\max_{0\le i\le t}(m_i)$. If Δ_t exceeds a shift threshold Υ, then shift is identified, triggering potential model adaptation. The shift threshold Υ is dynamically calculated as a factor of the mean of the historical change scores, represented as $\Upsilon_t=\kappa*\bar{m}_t$. Here, κ is a constant known as the 'shift-factor'. Consequently, lower κ increases potential adaptation frequency by relaxing shift evidence requirements, while higher κ reduces it [36]. This dynamic approach allows the threshold to adjust based on the average of change scores over time.

2) Robust Update Decision: The proposed robust update decision selectively determines when to update the model by leveraging information from latent-space shift identification and deviation detection, ensuring reliable and accurate updates. In particular, we incorporate a heuristic constraint that only uses new normal observations for updates. This protects the model from self-poisoning by anomalies, which could degrade its performance. The model update is triggered only when two conditions are met: 1) *significant shift detected:* the change intensity Δ_t must exceed the dynamic shift threshold Υ_t, and 2) *normal observation confirmed:* the deviation score s_t must be

Algorithm 2: Shift-Aware Model Adaptation (SMA).

1: Initialise $M, \bar{m}, m, \Delta$ to 0 ▷ Initialization at start
2: **procedure** MODEL_ADAPTATION($\boldsymbol{z}_r$, $\boldsymbol{z}_c$, s_t , η, ν ,κ)
3: Calculate change score $\delta_t = \mathcal{E}(\boldsymbol{z}_r, \boldsymbol{z}_c)$ using Eq. 17;
4: update $\bar{m}$ to include δ_t in the empirical average
5: $m_t = \max(m + \bar{m} + \delta_t - \nu, 0)$ ▷ Calculate new m_t
6: **if** $m_t > M$ **then**
7: $M_t = m_t$ ▷ Update maximum value
8: **end if**
9: $\Delta_t = M - m_t$ ▷ Compute the intensity of the current statistic
10: $\Upsilon_t = \kappa * \bar{m}$ ▷ Calculate shift threshold
11: **if** $\Delta_t > \Upsilon_t$ and $s_t > \eta$ **then**
12: $M = 0$, $\bar{m} = 0$, and $\Delta = 0$ ▷ Reset variables
13: return True ▷ Initiate model update
14: **else if** $\Delta_t > \Upsilon_t$ and $s_t \leq \eta$ **then**
15: return False ▷ Model update not performed
16: **else**
17: $M = M_t$ and $m = m_t$ ▷ No shift
18: return False
19: **end if**
20: **end procedure**

above the deviation threshold η, confirming that the current observation is likely normal. This ensures the model incorporates new knowledge from legitimate normality shifts while being robust to anomalies or transient fluctuations. However, in some cases, an update might be declined even if a shift is detected: *shift with anomaly:* if the change intensity is higher than the shift threshold $\Delta_t > \Upsilon_t$, but the deviation score s_t is less than or equal to the deviation threshold η (e.g., potential attack observations), the model update is declined. This measure acts as a safeguard, ensuring the model is not inadvertently updated with unreliable data, thereby preserving its integrity. The conditions for the model update process are summarised as follows:

$$\begin{cases} \Delta_t > \Upsilon_t \wedge s_t > \eta, & \text{Initiate model update} \\ \Delta_t > \Upsilon_t \wedge s_t \leq \eta, & \text{Decline model update} \end{cases} \tag{19}$$

The overall functioning of the Shift-aware Model Adaptation (SMA) is outlined in Algorithm 2. Suppose the change score Δ_t surpasses the shift threshold Υ_t and the deviation score s_t is above the deviation threshold η. In that case, SMA initiates a reset of its internal variables and triggers an update (Line 11). However, if the change score Δ_t exceeds the shift threshold Υ_t while the deviation score s_t is equal to or less than the deviation threshold η, the model update is declined (Line 14). For all other instances where no shift is detected, SMA merely refreshes its internal statistics and proceeds to process the incoming data without taking additional actions (Line 17).

3) Adaptive Regularisation Term: To prevent catastrophic forgetting, which occurs when a model forgets previously learned information during adaptation, we incorporate an adaptive regularisation term inspired by Bayesian online learning [37]. This term encourages the model to update its parameters gradually, building upon past knowledge while incorporating new information effectively. Our adaptive regularisation term achieves this by penalising large changes in the model's parameters. This ensures that the model builds upon its existing knowledge base rather than starting from scratch with each new data point. We specifically perform Bayesian inference over the emission parameter θ within our deep state space-based generative model because of its significant impact on detection performance. Specifically, the posterior distribution of the emission parameters learned from the previous sequence at time $t-1$ becomes the new prior distribution for the current sequence at time t. The posterior distribution $p(\boldsymbol{\theta}|\boldsymbol{x}_{1:T})$ after seeing T window of observations is recovered by applying Bayes' rule:

$$p(\boldsymbol{\theta}|\boldsymbol{x}_{1:T}) = p(\boldsymbol{\theta}) \prod_{t=1}^{T} p(\boldsymbol{x_t}|\boldsymbol{\theta}) \propto p(\boldsymbol{x}_T|\boldsymbol{\theta}) p(\boldsymbol{\theta}|\boldsymbol{x}_{1:T-1}). \tag{20}$$

where $p(\boldsymbol{x}_T|\boldsymbol{\theta})$ is the likelihood of the observations, and $p(\boldsymbol{\theta}|\boldsymbol{x}_{1:T-1})$ is the previous posterior of the parameters θ which serves as the new prior. This allows the model to build upon the learned information from past observations, effectively utilising the posterior distribution as a prior belief to guide the inference model for the current sequence. This can facilitate knowledge transfer from the past to the current data streams (i.e., forward transfer) [38]. Moreover, it can alleviate catastrophic forgetting by penalising drastic changes from the previously learned information. Often, the posterior distribution is intractable, necessitating approximation methods. We use the online variational inference (VI) [37] to approximate the posterior of (20) by a parametrised distribution $p(\boldsymbol{\theta}|\boldsymbol{x}_{1:T}) \approx \bar{q}(\boldsymbol{\theta})$ through a sequence of projections, which are defined via the minimisation of KL divergence over the set of allowed approximate posteriors Q:

$$\bar{q}(\boldsymbol{\theta}) = \arg\min_{q \in Q} D_{KL}\left(\bar{q}(\boldsymbol{\theta}) \parallel Z^{-1} \bar{q}_{t-1}(\boldsymbol{\theta}) p(x_t|\boldsymbol{\theta})\right) \tag{21}$$

where Z^{-1} is the normalisation constant. $\bar{q}(\boldsymbol{\theta})$ is an approximate posterior modelled as a multivariate Gaussian distribution. However, in the context of online variational inference, the variance of the Gaussian posterior approximation tends to shrink as new observations are incorporated over time. This can potentially lead to the loss of information from previous data streams, especially when learning from non-stationary and dynamic environments [26], [38]. Similar to [38], we integrate Bayesian exponential forgetting into the VI process as a forgetting factor, which assigns progressively less weight to older observations. This allows our model to adapt to new data while still retaining valuable knowledge from the past. The forgetting factor is incorporated into the update equation (20) for the approximate posterior, ensuring recent observations have a stronger influence while gradually forgetting older ones. This helps maintain a balance between adaptation and knowledge preservation, mitigating "posterior shrinkage" and enabling better continual learning. The Bayesian exponential forgetting is defined as follows:

$$\bar{q}_t(\boldsymbol{\theta}) = \bar{q}_{t-1}(\boldsymbol{\theta}) p(\boldsymbol{x}_t|\boldsymbol{\theta})^{(1-\epsilon)^{\Delta T/\tau}} \tag{22}$$

where ϵ is a small positive constant, and τ represents a time-constant corresponding to the mean value of the time-lags, expressed as $\Delta T = T - t$. By incorporating the forgetting factor, the posterior distribution places a relatively higher emphasis on the likelihood of recent observation, which allows the model to adapt to non-stationary data distributions and avoid excessive influence from past information.

To this end, we incorporate the adaptive regularisation term in (21) with the learning process of RDSAD, which is equivalent to

maximising the evidence lower bound (ELBO). We can modify the ELBO in (16) with respect to $\bar{q}$ and ϕ as follows:

$$\mathcal{L}(\phi, \bar{q}_t(\boldsymbol{\theta})) = \sum_{t=1}^{T} \Bigg[\mathbb{E}_{q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1},\boldsymbol{h}_t,\boldsymbol{x}_t)} \Bigg[\underbrace{\log p_\theta(\boldsymbol{x_t}|\boldsymbol{z}_t,\boldsymbol{h}_t)}_{\text{reconstruction term}} \Bigg] - \underbrace{D_{KL}\left(q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1},\boldsymbol{h}_t,\boldsymbol{x}_t) \parallel p_\theta(\boldsymbol{z}_t|\boldsymbol{z}_{t-1},\boldsymbol{h}_t)\right)}_{\text{KL-divergence term}} \Bigg] - \underbrace{D_{KL}\left(\bar{q}_t(\boldsymbol{\theta}) \parallel \bar{q}_{t-1}(\boldsymbol{\theta})\right)}_{\text{adaptive regularisation term}} \tag{23}$$

By incorporating the Bayesian adaptive regularisation with the ELBO objective, the model can adapt to the underlying change while retaining knowledge from previous observations. This approach enables efficient real-time learning and adaptation with improved generalisation performance [26], [38].

Overall, RDSAD's SMA mechanism specifically addresses the practical challenge of frequent normality shifts encountered in CPS environments. Our extensive experiments (Section VI-E) involving varying anomaly contamination levels confirm SMA's effectiveness in dynamically adapting to shifts while minimizing performance degradation. This confirms RDSAD's robust suitability for real-world CPS scenarios characterized by rapid and frequent environmental variations.

E. RDSAD Runtime Procedure

RDSAD is initially trained offline with the available normal dataset to optimise the model parameters and set the threshold settings. During the runtime, it actively monitors system behaviour for potential attack patterns. It also incrementally adapts to incoming data streams when change is identified, adjusting its internal structure to accommodate changes. Algorithm 3 outlines the overall runtime procedure of RDSAD. The algorithm begins by initialising the necessary parameters and data structures. Line 1 assigns the initial latent variables ($\boldsymbol{z}_p$) using values from the training phase. The sliding window ($\boldsymbol{W}$) and the variable ($\boldsymbol{z}_c$) maintaining the current latent variables are also initialised. As new observations arrive, they are incorporated into the sliding window. Once the window reaches its pre-set size (T), the algorithm iterates through each timestep. Lines 13-17 depict the computation steps to construct the inference and emission models at the current timestep, thereby capturing the underlying data dynamics. Line 19 updates $\boldsymbol{z}_c$ with the newly derived latent variable ($\boldsymbol{z}_t$), and the oldest latent variable is removed to maintain the window size. The likelihood score of observation $\boldsymbol{x}_t$ is calculated in Line 18. RDSAD then calculates the deviation score s (Line 21). If an update is triggered by the Shift-aware Model Adaptation (SMA) (see Algorithm 2), $\boldsymbol{z}_p$ is updated with $\boldsymbol{z}_c$ (Line 24), and the deviation threshold is recalculated (Line 25). Line 27 updates the parameters (ϕ, θ) using the Adam optimisation for an incremental model update. RDSAD continuously assesses the current observation $\boldsymbol{x}_t$ for deviation from the normal behaviour in Line 30. Aa alarm is raised if the deviation score s_t is less than or equal to the threshold η. Conversely, when s_t exceeds η (e.g., indicating normal behaviour), the statistics for consistent behaviour ($\bar{c}$) get updated (Line 34). RDSAD continually processes the data stream, removing the oldest observation from the sliding window (Line 39) and incrementing the timestep.

1) Computational Complexity: The computational complexity of RDSAD per input sequence is approximately $\mathcal{O}(T \cdot (D \cdot H + H^2))$, where T is the sequence length, D is the feature dimension, and H is the size of the latent GRU state. This complexity stems from the GRU-based modeling of latent dynamics (input-to-hidden and hidden-to-hidden transitions), along with amortized inference in the variational learning framework. Since these operations are applied linearly over time, RDSAD's complexity scales linearly with both sequence length and input dimensionality. Additionally, the Shift-aware Model Adaptation (SMA) module performs lightweight updates only when a significant concept drift is detected, introducing minimal computational overhead. These design choices ensure that RDSAD remains efficient and scalable for real-time deployment in streaming CPS environments. This theoretical analysis aligns with our empirical findings presented in Section VI-B.

IV. Related Work

This section discusses existing anomaly-based detection methods in industrial applications that have garnered significant attention in recent years. Specifically, we focus on two distinct approaches: static-based and streaming-based detection approaches. These approaches represent different paradigms for detecting malicious threats.

A. Static-Based Detection

Static-based anomaly detection methods operate by performing offline training on a batch of data and do not update their parameters during runtime. While static-based detection methods can operate in real time, they typically require frequent offline training or model updates to adapt to changing patterns in the data effectively.

Several static-based anomaly-based detection methods have been proposed in various domains [7], [13], [14], [15]. Deep learning-based anomaly detection, such as a deep one-class classification method, named DeepSVDD [13], and Variational Autoencoder (VAE) with a gating parameter β [14], have shown promise in capturing complex data patterns. Similarly, Zong et al. [15] leverage a deep Autoencoder (AE) with Gaussian Mixture Model (GMM), named DAGMM, to flag anomalies in low-density regions of the learned normal behaviour distribution. However, the aforementioned methods cannot capture time dependencies in sensor data.

In contrast, some recent works [16], [17] addressed some of these limitations by incorporating Long Short-Term Memory (LSTM) to capture temporal dependencies in sensor data, improving anomaly detection in sequential data. Malhotra et al. [16] propose LSTM-ED, a multi-sensor anomaly detection method based on LSTM. Furthermore, Park et al. [17] introduced LSTM-VAE, which integrates LSTM with VAE by

Algorithm 3: RDSAD Runtime.

Parameters: T, d, q,$\boldsymbol{z}_{\text{training}}$, $\boldsymbol{\ell}^*_{\text{training}}$, $\boldsymbol{s}_{\text{training}}$, ν, κ
Input: Data streams $\mathcal{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_t, \ldots\}$
Output: Raise an alarm whenever the deviation score crosses a threshold.

Initialise $\boldsymbol{z}_r$ with $\boldsymbol{z}_{t-T:t-1}$ from $\boldsymbol{z}_{\text{training}}$.
Initialise $\boldsymbol{z}_c$ with the last T latent variables as $\boldsymbol{z}_{t-T+1:t}$ from $\boldsymbol{z}_{\text{training}}$.
Initialise $\bar{c}$ from an initial subset $\boldsymbol{\ell}^*_{\text{training}}$
Initialise η using DeviationThreshold($\boldsymbol{s}_{\text{training}}$,$q$) based on training data $\boldsymbol{s}_{\text{training}}$.
Initialise a list $\boldsymbol{l}^*$ of size d
Initialise sliding window $\boldsymbol{W}$.
Set current time step $t = 0$.
while a new observation $\boldsymbol{x}_t$ arrives in the stream **do**
  Add $\boldsymbol{x}_t$ to $\boldsymbol{W}$.
  if window size $|\boldsymbol{W}|$ reaches T **then**
    for each observation $\boldsymbol{x}_t$ in $\boldsymbol{W}$ **do**
      Extract features $\varphi(\boldsymbol{x}_t)$ as in Eq. 5;
      Compute $\boldsymbol{h}_t$ as in Eq. 7;
      Compute $q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}, \boldsymbol{h}_t, \varphi(\boldsymbol{x}_t))$ as per Eq. 6;
      Compute $p_{\theta_z}(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}, \boldsymbol{h}_t)$ as per Eq. 8;
      Sample $\boldsymbol{z}_t \sim g_\phi(\boldsymbol{x}_t, \boldsymbol{\epsilon})$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I})$;
      Compute $p_\theta(\boldsymbol{x}_t|\boldsymbol{z}_t, \boldsymbol{h}_t)$ as in Eqs. 10 & 11;
      Calculate ℓ_t as in Eq. 12
      Add $\boldsymbol{z}_t$ to $\boldsymbol{z}_c$, and if $|\boldsymbol{z}_c| > T$, remove $\boldsymbol{z}_{t-T}$
    end for
    Compute $s_t = \bar{c} - \ell_t$ as in Eq. 13a;
    /* Shift-aware Model Adaptation (SMA) */
    if model_adaptation($\boldsymbol{z}_r$, $\boldsymbol{z}_c$, s_t , η, ν, κ) **then**
      Update $\boldsymbol{z}_p \leftarrow \boldsymbol{z}_c$
      $\eta \leftarrow$ DeviationThreshold($\boldsymbol{s}_C$,q)
      /* Update ϕ, θ according to Eq. 23 */
      $\phi, \theta \leftarrow \text{Adam}(-\nabla_{\theta,\phi}\mathcal{L}(\phi, \bar{q}_t(\theta), \boldsymbol{W}))$;
    end if
    /* Deviation Detection */
    if $s_t \leq \eta$ **then**
      Raise an alarm for deviation cases.
    else if $s_t > \eta$ **then** ▷ In the normal case
      add ℓ_t to $\boldsymbol{l}^*$.
      Update $\bar{c}$ as the mean of the elements in $\boldsymbol{l}^*$ as in Eq. 13b.
      if $|\boldsymbol{l}^*| > d$ **then**
        remove the oldest element.
      end if
    end if
    Remove oldest observation from window $\boldsymbol{W}$.
  end if
  Increment time step t.
end while

replacing the feed-forward network in a VAE with LSTM. Nevertheless, these methods also have their constraints, in capturing the inherent variability and stochastic nature of sequential data streams. In the context of sequential data modelling, incorporating the temporal dependencies of stochastic variables is crucial for accurately representing the underlying dynamics of the input data streams [8], [9].

Recently, advanced static-based approaches leverage graph-based methods, and digital-twin paradigms have emerged. Deng et al. [39] introduced the Graph Deviation Network (GDN), a method exploiting graph structures to effectively model feature relationships in multivariate time-series. Yang et al. [40] proposed Agent-based Dynamic Thresholding (ADT), employing an adaptive dynamic thresholding approach to enhance robustness against evolving anomaly patterns. Similarly, digitaL twin-based Anomaly deTecTion wIth Curriculum lEarning (LATTICE) [41] leveraged digital-twin methodologies to enhance anomaly detection in CPS environments, effectively managing shifts through adaptive strategies. However, despite their advancements, these static methods still face limitations, particularly their inability to handle temporal uncertainty effectively or proactively manage concept drift, self-poisoning, and catastrophic forgetting.

Overall, the performance of static-based detection methods can degrade when faced with significant changes in the underlying data patterns. They often require manual intervention to update the model parameters, which can be time-consuming and not feasible in dynamic environments. This deficiency restricts their ability to promptly react to new or evolving threats, potentially compromising system security [23], [25], [26], [27].

B. Streaming-Based Detection

Unlike static-based anomaly methods, streaming-based anomaly detection methods enable real-time adaptation to evolving patterns in streaming data without frequent offline training [23], [25], [26]. Several popular streaming anomaly detection methods exist in the literature. Methods like STORM [21] and HS-Tree [22] utilise sliding window approaches for anomaly detection in streaming data, focusing on global distance-based anomalies and leveraging tree-based structures, respectively. Pevny et al. [24] present an ensemble-based approach called LODA that leverages histograms based on random projections to generate anomaly scores. Some works have utilised Deep Learning (DL) methods to propose online and streaming anomaly detection. Mirsky et al. propose KitNET [25], an online unsupervised deep learning approach that employs an ensemble of autoencoders for cyber-attack detection. However, these methods do not consider the time dependencies in the data streams. In contrast, Saurav et al. [23] develop Online-RNN-AD, a deep learning method for online time series anomaly detection that leverages Recurrent Neural Networks (RNNs) to learn the temporal dependencies in the data streams. Moreover, Hartl et al. [42] introduced SDOstream, a streaming anomaly detection approach based on low-density models designed for real-time outlier detection. While effective in detecting anomalies in a streaming context, SDOstream lacks explicit mechanisms for handling temporal dependencies, uncertainty, and dynamic shifts.

Despite their advancements, these streaming-based methods lack effective and shift-aware solutions that can proactively update the model when changes are detected in the underlying data patterns. Furthermore, they often exhibit some challenges like self-poisoning and catastrophic forgetting, hindering their ability to maintain model integrity over time and recognise previously learned normal behaviours. While memory-based solutions from online and continual learning literature like pseudo-rehearsal methods [43] and experience replay [27] offer potential mitigation, their primary application in supervised learning scenarios limits their direct applicability to unsupervised intrusion detection in industrial CPS [26], [27].

Table II provides a brief comparison between RDSAD and existing static and streaming methods. In contrast to prior research, RDSAD explicitly considers both the uncertainty and temporal dependencies in data streams. This allows RDSAD to capture essential information from the data streams, leading to improved

TABLE II
A COMPARISON OF EXISTING SOLUTIONS

	Method	Temporal uncertainty modelling	Shift-aware	Self-poisoning	Catastrophic forgetting
Static-based	DeepSVDD [13]	○	○	○	○
	Beta-VAE [14]	○	○	○	○
	DAGMM [15]	○	○	○	○
	LSTM-ED [16]	○	○	○	○
	LSTM-VAE [17]	◐	○	○	○
	GDN [39]	○	○	○	○
	ADT [40]	○	◐	○	○
	LATTICE [41]	○	◐	○	○
Streaming-based	STORM [21]	○	○	○	○
	HS-Tree [22]	○	○	○	○
	Online RNN-AD [23]	○	○	○	○
	LODA [24]	○	○	○	○
	KitNET [25]	○	○	○	◐
	SDOstream [42]	○	○	○	○
	RDSAD	●	●	●	●

(●: fully satisfy a criterion, ◐: partially satisfy, ○: do not satisfy a criterion).

TABLE III
SUMMARY OF THE DATASETS. THE ANOMALIES PERCENTAGE WITHIN THE TEST SAMPLES

Dataset	Train size	Test size	Dimensions	Anomalies (%)
ToN_IoT [44]	245,000	160,447	19	38.92
SWaT [45]	475,200	449,919	51	11.91
WADI [46]	784,571	172,801	123	5.77
GAS [47]	12,269	13,910	128	2.73

inference and more effective detection of deviation and changes. Furthermore, RDSAD incorporates a shift-aware update approach, selectively updating the model when significant changes in data patterns are identified. It also employs robust update decisions to prevent self-poisoning during the update process, preserving the model's integrity. Finally, RDSAD introduces an adaptive regularisation term that mitigates catastrophic forgetting in an unsupervised setting by constraining the parameter values to not change significantly from their previously learned value. Unlike memory-based methods, RDSAD does not rely on memory architectures or storage of prior data streams. By avoiding the need for storage of data, RDSAD reduces resource demands and significantly mitigates potential data privacy concerns, making it a more effective and privacy-preserving solution.

V. EXPERIMENT DESIGN

Extensive experiments were carried out to compare the performance of RDSAD with those of the state-of-the-art methods. In what follows, we describe the datasets, benchmark methods, implementation settings, as well as the evaluation metrics used in the experiments.

A. Datasets

Four benchmark datasets, well-established in security research, are used for the experiments: TON_IoT [44], Secure Water Treatment (SWaT) [45], Water Distribution Dataset (WADI) [46], and GAS [47]. Table III provides a summary of the characteristics of each dataset. Descriptions of each dataset. The TON_IoT dataset comprises data streams collected from seven various IoT/IIoT services (e.g., weather and temperature) functioning on a medium-scale IoT network. The SWaT data originates from a large-scale water treatment plant replica located at the Singapore University of Technology and Design (SUTD). It was collected over 11 days, capturing the statuses and readings from 51 sensors and actuators. The WADI dataset contains measurements from 123 sensors and actuators gathered from the water distribution testing environment at SUTD University. Finally, the GAS dataset was collected over 36 months from several chemical sensors installed in a gas delivery system. The objective is to keep track of six various types of gaseous substances. The first three datasets contain anomalies representing diverse security threats, including cyber-attacks designed to disrupt the availability and integrity of the testbed systems where the datasets originated. In the GAS dataset, the presence of Acetaldehyde, a harmful chemical, is considered a threat pattern. All datasets were labelled by domain experts, and anomalies are exclusively found within the test data.

B. Benchmark Methods

Several state-of-the-art methods are used as baselines to evaluate the performance of RDSAD. A selection of both static and streaming methods is considered to ensure a thorough comparison in the anomaly detection context. The static-based methods include: DeepSVDD [13], Beta-VAE [14], DAGMM [15], LSTM-ED [16], and LSTM-VAE [17], (see Section IV-A). The streaming-based methods comprise: STORM [21], HS-Tree [22], Online RNN-AD [23], LODA [24], and KitNET [25], (see Section IV-B).

C. Implementation Settings

We implemented RDSAD using Python (version 3.7.14) and Pytorch (version 1.12.1). The machine on which RDSAD and all baseline methods are executed comes with an integrated Intel Xeon, E5-1650 CPU, and 32.0 GB RAM, operating on a Windows 10 OS. To ensure reproducibility and fair comparison, detailed implementation settings and baseline parameters are available in Appendix A, available online.

D. Parameters Settings

Here, we discuss the parameter settings for RDSAD. We empirically set the window size to $T = 50$ for GAS, $T = 100$ for SWaT and WADI and $T = 150$ for ToN_IoT, respectively. The Temporal Embedding Network φ is configured with a TCN model with two blocks of convolutional layers with ReLU activation function. Each block has two temporal convolutional layers with a kernel size of $k = 3$. Moreover, the inference and emission models are configured with two fully connected layers. The inference model is structured with a first layer containing 32 neurons and a second layer comprising 64 neurons. Conversely, the emission model is constructed with 64 neurons in its first layer and 32 in the second. The GRU-flow is configured with 64 units. We observed that the chosen architecture of this model provided sufficient capacity to achieve improved performance. However, incorporating additional capacity yielded minimal benefits while simultaneously increasing model sizes and runtime. Furthermore, the scalar parameters α_u and α_r

of GRU-FLOW are set to their default values 0.4 and 0.8, respectively. The Tanh activation function $tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ is utilised in GRU-flow due to its computational efficiency. We employ ℓ_2-norm regularisation with a coefficient of 10^{-6} across all layers as a measure to avoid model overfitting. The dimension of the z-space is empirically determined and set to 4 Section VI-G). For the deviation threshold, the value of q is typically chosen within the range of 10^{-3} and 10^{-5} to allow for some flexibility and accommodate potential errors in setting the value of q. In our experiment, we set q to (e.g., 10^{-4}) for all datasets to achieve a high true positive rate (i.e., accurate deviation detection) while keeping a low false positive rate (i.e., minimising incorrect detections). The low quantile is set to 0.01 for ToN_IoT, 0.05 for both SWaT and WADI and 0.07 for GSA-Drift. For the adaptive regularisation term, We construct a Bayesian network over θ parameters using a fully-connected perceptron consisting of two layers. Each layers incorporates 100 hidden units, which employ ReLU activation functions. We use a Gaussian mean-field approximation over each of the parameters θ. We place a standard normal Gaussian over each parameter as the initial prior $(p_0(\theta))$. This choice reflects our initial belief about the parameter values before any data streams are observed. We set the parameter ϵ for Bayesian exponential forgetting to $\epsilon = 0.05$, as in the original paper. The coefficient λ controlling the regularisation in SMA (i.e., penalizing deviation from past latent encodings) was fixed at 0.1in our experiments. This value was selected based on preliminary runs using a grid search over $\lambda \in 0.01, 0.05, 0.1, 0.5$, where 0.1 yielded the most stable trade-off between plasticity (i.e., adaptation to new concepts) and stability (i.e., memory of past normality patterns). Lastly, we apply Adam optimizer for stochastic gradient descent [29], setting an initial learning rate of 10^{-3} to optimise the model parameters of both RDSAD (θ and ϕ) and the Bayesian network.

The initial average training time of our model for the ToN_IoT, SWaT, WADI, and GAS datasets is approximately 52, 45, 68, and 30 minutes, respectively, on an Intel Xeon E5-1650 CPU with 32 GB RAM (Section V-C). Training times scale linearly with data size, confirming computational efficiency for large CPS deployments.

E. Evaluation Metrics

The performance of RDSAD and benchmark methods is evaluated using precision, recall, F-score, and the Area Under the Receiver Operating Characteristic Curve (AUC-ROC). *Precision* $= \frac{TP}{(TP+FP)}$ represents the proportion of correctly detected anomalies among all predicted anomalies. *Recall* $= \frac{TP}{(TP+FN)}$ is the ratio of correctly detected anomalies to the total number of actual anomalies. *F-score* $= 2 * \frac{(Recall * Precision)}{Recall + Precision}$ calculates the harmonic mean of precision and recall [3]. The term *TP* represents the number of true-positives predictions, and *FP* refers to false-positives whereas *FN* denotes the number of false-negatives predictions. *AUC-ROC* measures the ability of the model to distinguish true anomalies from normal data at various thresholds. Higher AUC-ROC values indicate better overall performance, with 1 being perfect and 0.5 representing random chance [3].

VI. Results

This section discusses the experimental results obtained from evaluating RDSAD on various aspects: overall performance, scalability, robust updates, ablation study, and parameter sensitivity.

A. Overall Performance

Table IV provides the precision, recall, F-score, and AUC for our proposed RDSAD as well as the baseline methods applied across four public datasets: ToN_IoT, SWaT, WADI and GAS. We conducted five experimental runs for each dataset and reported the average and standard deviation ($\pm$) of the results. The highest-performing scores have been highlighted in bold for easy reference. RDSAD consistently achieves the highest results across all the benchmark datasets. Specifically, it obtains the highest precision values of 0.7071 and 0.6157 on the ToN_IoT and GAS datasets, respectively. While GDN achieves exceptional precision of 0.9935 on SWaT, it has a lower recall of 0.6812, indicating potential challenges in detecting all anomalies in certain contexts. In contrast, RDSAD achieves an optimal balance with high precision (0.9175), recall (0.8552), and the highest F-score (0.8851) and AUC (0.9006). ADT Shows balanced results across datasets, achieving an F-score of 0.6296 on ToN_IoT and 0.8043 on SWaT. On the WADI dataset, RDSAD has the second-best precision score of 0.8316 compared to LSTM-ED, which has the best precision of 0.9239. In addition, RDSAD achieves the best F-score with values of 0.7628 and 0.8851 on the ToN_IoT and SWaT datasets, as well as 0.8474 and 0.6686 on WADI and GAS datasets, respectively. Moreover, RDSAD outperforms all baseline methods in terms of AUC metric with the highest values of 0.8112, 0.9006, 0.8555 and 0.7861 on ToN_IoT, SWaT, WADI, and GAS datasets, respectively. These results highlight the overall effectiveness and robustness of RDSAD in various scenarios.

Even though LODA obtains the highest recall values across the majority of datasets, it exhibits a notable low precision. Specifically, on the SWaT dataset, LODA achieves a recall score of 0.9860 but with the lowest precision at 0.1227. Similarly, it demonstrates the best recall values on ToN_IoT and WADI datasets, at 0.9840 and 0.9649, respectively, but these are contrasted with low corresponding precision scores of 0.3897 and 0.0575. This is attributed to the inherent limitation of LODA, where the employed random projection fails to preserve the accurate pairwise distances of the original space. As a result, LODA may not provide precise outlier estimation due to this constraint. LATTICE demonstrates moderate performance, achieving competitive F-scores (e.g., 0.7642 on WADI), but its imbalance between precision and recall, particularly on ToN_IoT and GAS datasets, reveals constraints in handling diverse anomaly patterns. Similar insight was observed in other methods. For example, KitNet has a recall of 0.9248 and 0.6520 with low precision of 0.2836 and 0.2795 on SWaT and GAS datasets, respectively. DeepSVD scores 0.8026 in a recall but with a precision of 0.2757 on the SWaT datasets. Similarly, DAGMM obtains a recall of 0.8749 with the second-lowest precision of 0.2125 On the SWaT datasets. Moreover, HS-Tree

TABLE IV
COMPARISON OF PRECISION, RECALL, F-SCORE, AND AUC METRICS FOR RDSAD AND BASELINE METHODS ON THE BENCHMARK DATASETS, PRESENTED WITH ± STANDARD DEVIATION

	Method	ToN_IoT				SWaT			
		Precision	Recall	F-score	AUC	Precision	Recall	F-score	AUC
Static	DeepSVDD	0.5005±0.0549	0.1944±0.1604	0.2391±0.1026	0.5198±0.0094	0.2757±0.0442	0.8026±0.0303	0.4075±0.0405	0.7486±0.0159
	Beta-VAE	0.3723±0.0393	0.1421±0.1604	0.1756±0.1406	0.5060±0.0227	0.8182±0.0022	0.6733±0.0014	0.7387±0.0017	0.8262±0.0008
	DAGMM	0.5064±0.0665	0.6093±0.0467	0.5536±0.0603	0.6143±0.0713	0.2125±0.0123	0.8749±0.0224	0.3417±0.0164	0.7203±0.0183
	LSTM-VAE	0.4694±0.0002	0.6969±0.0004	0.5609±0.0003	0.5959±0.0003	0.8109±0.0003	0.6611±0.0002	0.7284±0.0002	0.8198±0.0001
	LSTM-ED	0.6445±0.0068	0.2970±0.0031	0.4066±0.0043	0.5960±0.0026	0.8151±0.3116	0.6645±0.2541	0.7321±0.2799	0.8217±0.1448
	GDN	0.7173±0.0018	0.7845±0.0016	0.7493±0.0017	0.7943±0.0025	**0.9935±0.0033**	0.6812±0.0013	0.8082±0.0006	0.8801±0.0005
	ADT	0.5950±0.0010	0.6685±0.0008	0.6296±0.0019	0.6312±0.0010	0.7312±0.0013	0.8932±0.0021	0.8043±0.0015	0.8710±0.0006
	LATTICE	0.6814±0.0011	0.4736±0.0009	0.5588±0.0010	0.5659±0.0012	0.7885±0.0031	0.7857±0.0019	0.7870±0.0011	0.8540±0.0018
Streaming	STORM	0.4018±0.0000	0.4125±0.0000	0.4071±0.0000	0.5102±0.0000	0.5649±0.0019	0.4606±0.0015	0.5074±0.0017	0.7055±0.0009
	HS-Tree	0.3920±0.0003	0.4912±0.0278	0.4357±0.0117	0.5024±0.0004	0.2599±0.0005	0.6357±0.0013	0.3690±0.0007	0.6913±0.0007
	LODA	0.3897±0.0000	**0.9840±0.0320**	0.5582±0.0054	0.5000±0.0000	0.1227±0.0000	**0.9860±0.0281**	0.2182±0.0006	0.5001±0.0002
	Online-RNN-AD	0.4900±0.0210	0.1257±0.0054	0.2001±0.0086	0.5211±0.0044	0.7765±0.0319	0.633±0.0260	0.6974±0.0287	0.8038±0.0148
	KitNet	0.3886±0.0000	0.3988±0.0000	0.3936±0.0000	0.4990±0.0000	0.2836±0.0000	0.9248±0.0000	0.4341±0.0000	0.7991±0.0000
	SDOstream	0.5582±0.0016	0.4707±0.0012	0.5107±0.0015	0.4180±0.0018	0.6951±0.0015	0.7662±0.0013	0.7289±0.0010	0.8202±0.0012
	RDSAD	**0.7071±0.0005**	0.8278±0.0008	**0.7628±0.0006**	**0.8112±0.0011**	0.9175±0.0005	0.8552±0.0039	**0.8851±0.0009**	**0.9006±0.0023**

	Method	WADI				GAS			
		Precision	Recall	F-score	AUC	Precision	Recall	F-score	AUC
Static	DeepSVDD	0.4387±0.0294	0.4641±0.0512	0.4512±0.0374	0.6601±0.0272	0.6068±0.0000	0.4916±0.0000	0.5431±0.0000	0.7617±0.0000
	Beta-VAE	0.2365±0.0057	0.4119±0.0099	0.3004±0.0072	0.6654±0.0052	0.4125±0.0022	0.3009±0.0021	0.3478±0.0018	0.6414±0.0009
	DAGMM	0.6285±0.0252	0.5781±0.1333	0.6022±0.0423	0.6475±0.0699	0.1469±0.0171	0.2056±0.0239	0.1714±0.0199	0.5146±0.0137
	LSTM-VAE	0.2821±0.0196	0.3442±0.0250	0.3101±0.0130	0.6504±0.0027	0.3617±0.0040	0.4385±0.1285	0.3892±0.0417	0.6620±0.0468
	LSTM-ED	**0.9239±0.0009**	0.7288±0.0007	0.8149±0.0008	0.7624±0.0062	0.4327±0.0039	0.6056±0.0054	0.5048±0.0045	0.7442±0.0031
	GDN	0.9150±0.0043	0.6019±0.0035	0.7261±0.0032	0.7320±0.0070	0.6850±0.0009	0.6319±0.0013	0.6573±0.0010	0.7450±0.0012
	ADT	0.8067±0.0010	0.7784±0.0009	0.7922±0.0007	0.8182±0.0009	0.6107±0.0015	0.6413±0.0013	0.6256±0.0012	0.7280±0.0014
	LATTICE	0.7924±0.0011	0.7380±0.0010	0.7642±0.0009	0.8015±0.0010	0.5981±0.0017	0.6237±0.0014	0.6106±0.0015	0.7053±0.0018
Streaming	STORM	0.0387±0.0007	0.0674±0.0012	0.0492±0.0009	0.4827±0.0006	0.1508±0.0053	0.4687±0.0166	0.2281±0.0081	0.5394±0.0095
	HS-Tree	0.1065±0.0011	0.4637±0.0047	0.1732±0.0017	0.6134±0.0025	0.1853±0.0175	0.5760±0.0545	0.2804±0.0265	0.6009±0.0313
	LODA	0.0575±0.0003	**0.9649±0.0702**	0.1086±0.0000	0.5010±0.0021	0.1338±0.0096	0.7263±0.4150	0.1988±0.0637	0.5160±0.0087
	Online-RNN-AD	0.1258±0.0166	0.2191±0.0289	0.1598±0.0211	0.5631±0.0153	0.4597±0.0996	0.3580±0.0776	0.4025±0.0872	0.6479±0.0445
	KitNet	0.2709±0.0002	0.4721±0.0004	0.3443±0.0003	0.6974±0.0002	0.2795±0.0012	0.6520±0.0028	0.3913±0.0017	0.7019±0.0016
	SDOstream	0.6421±0.0018	0.6954±0.0016	0.6676±0.0013	0.7131±0.0015	0.4808±0.0022	0.5729±0.0020	0.5228±0.0018	0.6422±0.0020
	RDSAD	0.8316±0.0032	0.8632±0.0057	**0.8474±0.0051**	**0.8555±0.0032**	**0.6157±0.0004**	**0.7312±0.0014**	**0.6686±0.0004**	**0.7861±0.0150**

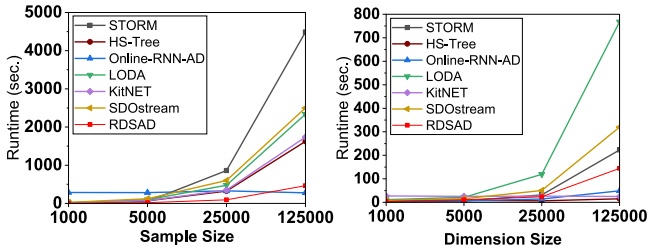
shows similar fluctuations in precision and recall on SWaT and GAS datasets. These findings indicate that these methods can successfully detect a significant proportion of abnormal observations. However, their low precision suggests that many of the detected observations are incorrect, resulting in a high number of false positives. This limitation might stem from the failure of these methods to adapt and update their internal structures to capture the evolving patterns in the streaming data, leading to a higher number of false positives and lower precision.

In contrast, while some methods have high precision, they often show relatively low recall scores. For example, LSTM-ED demonstrates a precision value of 0.6445, yet it has a low recall of 0.2970 on the ToN_IoT dataset. DeepSVD and Online-RNN-AD have precision values of 0.5005 and 0.4900 with low recall scores of 0.1944 and 0.1257 on the ToN_IoT dataset. Furthermore, Beta-VAE, LSTM-VAE and LSTM-ED show good precision values and have relatively low recall scores on the SWaT dataset. This arises because these methods accurately identify only a limited number of abnormal observations. However, they fail to detect most abnormal instances, leading to a low recall rate. This reduced performance may be attributed to the inability of these methods to accurately identify certain anomalies and their lack of handling uncertainties inherent in the data streams, which further contribute to the lower recall rate. Finally, SDOstream achieves an F-score of 0.5107 on ToN_IoT and 0.7289 on SWaT, indicating reasonable detection capabilities, particularly in streaming data contexts. However, RDSAD consistently outperforms SDOstream and other streaming methods, highlighting RDSAD's ability to handle real-time adaptive scenarios robustly.

RDSAD exhibits a minimal disparity between precision and recall, indicating its superior robustness compared to other methods. It shows a balanced performance in all evaluation metrics, highlighting its ability to effectively identify anomalies while simultaneously minimising false positives and false negatives. This close alignment between the metrics signifies that our model excels in its ability to strike a balance, thereby ensuring robust deviation detection performance. RDSAD is designed with a Shift-aware Model Adaptation (SMA) that enables it to dynamically adapt and update its internal structures in real time, effectively capturing the evolving patterns in the streaming data. This adaptability allows it to maintain a balance between detecting anomalies and minimising false positives. Furthermore, RDSAD leverages a deep state-space model and incorporates a t-student distribution, which can effectively consider uncertainty and capture the underlying patterns hidden within the noisy data.

B. *Scalability*

This section presents the scalability of RDSAD and various streaming methods in terms of runtime performance across varying sample sizes and dimensions. To conduct these experiments, the DataStream module was utilised in scikit-multiflow [48] to generate synthetic datasets. The sample size experiments were conducted on four synthetic datasets, each consisting of 100 dimensions and varying sample sizes of $1000, 5000, 25000$, and 125000. Similarly, the scale-up experiments were performed on four synthetic datasets with a fixed sample size of 500, while the dimensions varied in a range of $1000, 5000, 25000$, and 125000.



(a) Sample (b) Dimension

Fig. 4. Runtime in relation to: (a) various sample sizes and (b) a different number of dimensions.

Fig. 4 illustrates the runtime associated with the sample and dimension experiments. The overall results illustrate that RDSAD consistently demonstrates linear and proportional increases in runtime in relation to both sample size and dimension. Regarding sample size experiments, RDSAD demonstrates superior runtime efficiency across various sample sizes compared to other streaming methods, as illustrated in Fig. 4(a). Online-RNN-AD exhibits the second-best runtime performance. HS-Tree and KitNet exhibit moderate runtime performances, with SDOstream closely following LODA due to their similar underlying approach. STORM experiences the worst runtime due to the computational burden of processing larger sample sizes. The increased number of data points necessitates extensive distance calculations and detection operations, leading to a notable increase in computational time. For the scale-up experiments, the runtime performance of RDSAD and the streaming methods is depicted in Fig. 4(b). Notably, all methods, except for LODA, demonstrate competitive runtimes between 1000 and 25000. This enhanced efficiency can be attributed to integrating dimensionality reduction within these methods, contributing to faster runtime execution. Both STROM and RDSAD have a slight increase in runtime as the dimensions extend from 25000 to 125000, while the remaining methods demonstrate consistent and stable performance. In contrast, LODA has the worst runtime among the other methods. As the dimensions of the data increase, LODA's computational complexity escalates due to the generation of random projections and the computation of density estimation histograms for each projection, resulting in higher computational costs.

Our empirical scalability evaluations (Fig. 4) clearly indicate the feasibility and practicality of deploying RDSAD in real-world industrial CPS scenarios involving varying data complexities and sizes. Specifically, RDSAD demonstrated linear computational complexity, ensuring practical scalability with larger datasets and higher dimensional data streams.

C. Computational Cost Analysis

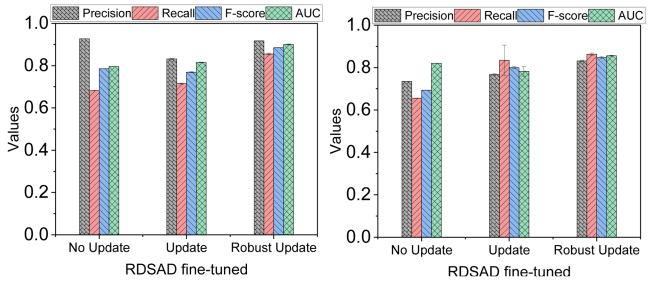
In addition to detection accuracy, the practical deployment of anomaly detection models in cyber-physical systems (CPSs) requires efficiency in terms of training time and memory usage. This section provides a comparative analysis of the computational cost of RDSAD against a subset of representative static and streaming baseline methods. Particularity, we selected six methods to represent a diversity of architectural approaches, including graph neural networks (GDN), reinforcement learning-based models (ADT), online learning (KitNET, SDOstream), and hybrid deep generative models (DAGMM, LSTM-VAE).

TABLE V
TRAINING TIME AND ESTIMATED MEMORY USAGE ON SWAT DATASET

Method	Training Time (mins)	Memory Usage (MB)
DAGMM	120	980
LSTM-VAE	90	1100
GDN	55	960
ADT	62	1010
KitNET	38	690
SDOstream	29	620
RDSAD	45	750

We measure the average training time (in seconds) and estimate memory usage (in megabytes) using PyTorch's built-in memory profiler, evaluated on the SWaT dataset.

As shown in Table V, RDSAD achieves competitive training time (45 s) with moderate memory requirements (750 MB), outperforming many deep learning-based approaches (e.g., DAGMM and LSTM-VAE) while maintaining higher detection performance. Unlike methods that rely on large encoder-decoder architectures, RDSAD uses a compact generative model with adaptive update mechanisms that minimize retraining and reduce memory pressure. While streaming baselines (KitNET, SDOstream) are faster and they have lower memory usage, this is justified by RDSAD's advanced features (e.g., uncertainty modeling, concept drift adaptation) that offer an effective balance between runtime efficiency and detection robustness.



(a) SWaT dataset (b) WADI dataset

Fig. 5. Retraining effect on RDSAD's performance.

D. The Effect of Robust Update Decision

This section discusses the influence of the proposed robust update decision on the overall accuracy of RDSAD using SWaT and WADI datasets, as they are widely used in security research. Three update scenarios are considered: (1) without any update, (2) with a regular update, and (3) with a proposed robust update decision. The objective is to evaluate how these update methods affect the overall performance of RDSAD. Fig. 5 presents the results for both datasets. The robust update decision consistently surpasses the performance of the other two scenarios. This can be attributed to its ability to selectively decline updates during potential anomalies, preventing their negative influence on the

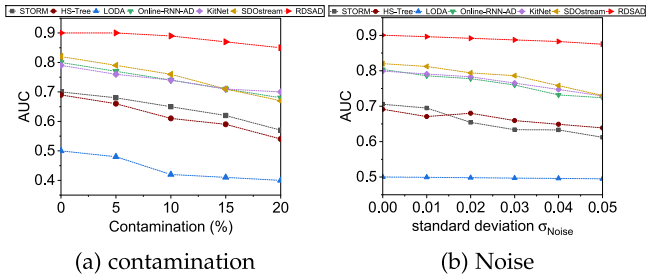


Fig. 6. AUC performance under varying contamination and noise levels on SWaT dataset.

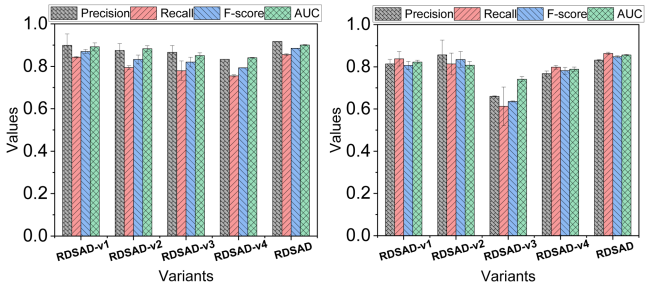


Fig. 7. Impact of various components of RDSAD.

model and mitigating self-poisoning. Thus, it plays an indispensable role in countering the effects of anomalies, enhancing RDSAD's overall performance significantly.

E. Robustness Under Extreme Conditions

To evaluate the robustness of RDSAD under extreme scenarios, we conducted two additional experiments: (I) contamination with anomaly samples and (II) additive Gaussian noise injection, as depicted in Fig. 6.

1) Anomaly Contamination: We injected varying levels of abnormal data into the training set of the SWaT dataset at contamination rates of 0%, 5%, 10%, 15%, and 20%. As illustrated in Fig. 6(a), RDSAD maintains a high AUC and shows minimal performance degradation compared to other streaming methods. Methods like STORM and LODA degrade rapidly (AUC drops to 0.59 and 0.40, respectively). RDSAD's Shift-aware Model Adaptation (SMA) mitigates self-poisoning by selectively updating the model only during confirmed normality shifts, preserving detection accuracy. This highlights its robustness in self-supervised settings where perfect separation between normal and abnormal data may not be feasible.

2) Gaussian Noise Injection. We injected additive Gaussian noise into SWaT sensor readings as it is very common in sensor measurements, using zero-mean noise with varying standard deviation $\sigma \in [0.0, 0.05]$. Fig. 6(b) shows that RDSAD maintains stable performance (AUC $= 0.8782$ at $\sigma = 0.05$), degrading by only 2.5% from baseline. This robustness of RDSAD is attributed to its probabilistic modeling and dynamic adaptation features. Nonetheless, KitNet and SDOstream suffer significant drops (AUC declines by 5–7%) due to their reliance on static feature assumptions.

F. Ablation Study

To assess the impact of the main components of RDSAD on its overall performance, an ablation study was conducted by removing one component at a time. This resulted in the creation of four distinct variants of RDSAD, namely: *RDSAD-v1*, *RDSAD-v2*, *RDSAD-v3* and *RDSAD-v4*. In the first variant, *RDSAD-v1*, the TCN component was omitted and replaced with a regular RNN. The second variant, *RDSAD-v2*, replaces the GRU-flow with a basic GRU. In *RDSAD-v3*, the regularisation term in (23) was removed from the configuration of RDSAD. Lastly, in *RDSAD-v4*, a Normal distribution is employed as emission instead of student-t distribution. This evaluation allows for a comprehensive understanding of the individual contributions of these components to the performance of RDSAD.

Fig. 7 depicts the comparative performance of RDSAD and its variants. The integration of each component into RDSAD significantly enhances the overall performance. In particular, RDSAD is slightly better than *RDSAD-v1*, which is based on RNNs. Compared to a regular RNN, RDSAD leverages TCN that utilises dilated convolutions, allowing for a larger receptive field and capturing information from a broader temporal context. This enables TCNs to model complex temporal relationships and dependencies more effectively than regular RNNs. Moreover, RDSAD outperforms *RDSAD-v2* because a basic GRU cannot capture the continuous evolution dynamics of latent states within evolving data streams. Furthermore, RDSAD exhibits superior performance compared to *RDSAD-v3*, which lacks an adaptive regularisation term. The absence of this term in *RDSAD-v3* leads to catastrophic forgetting, losing past knowledge during updates. This highlights the importance of the adaptive term in RDSAD, as it preserves knowledge and contributes to its improved performance. Finally, RDSAD achieves better results than *RDSAD-v4* with Normal distribution. This suggests that introducing a heavy-tailed distribution can enhance the robustness of the overall performance.

G. Parameter Sensitivity

This section investigates the performance of RDSAD with respect to variations in the dimension of latent z-space, window length, and shift factor. The goal is to examine how these parameters impact the effectiveness of RDSAD on the selected datasets. The F-score of RDSAD is examined while varying the latent z-space dimension within a range of $\{2, 32\}$, as depicted in Fig. 8(a). It is observed that the results consistently demonstrate that the highest F-score is achieved when the dimension of z is set to 4 for all datasets, and the performance stabilises beyond this dimension. Consequently, a dimension of 4 is selected for the z-space representation. To evaluate the influence of window length, we conducted experiments using different window lengths, specifically $T = [50, 100, 150, 200, 250]$. Fig. 8(b) illustrates the results corresponding to these five different window lengths. RDSAD demonstrates a good performance with a window length of $T = 100$ for both the SWaT and WADI datasets.

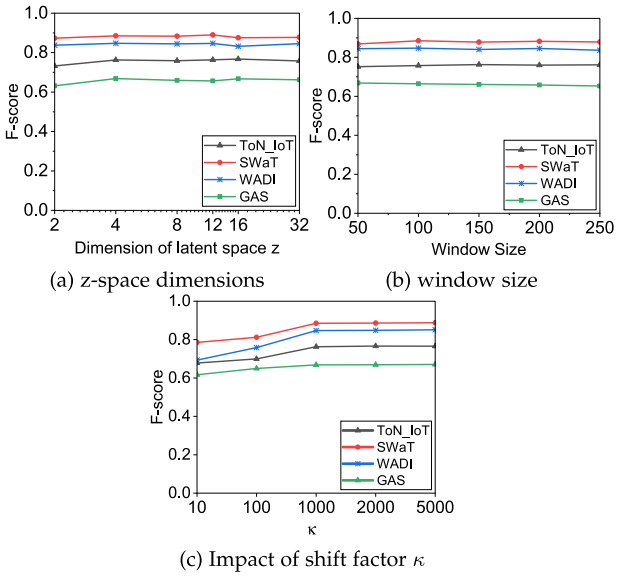


Fig. 8. Parameter sensitivity of RDSAD.

Similarly, it achieves satisfactory results with a window length of $T = 150$ for the ToN_IoT dataset and $T = 50$ for the GAS dataset. Overall, the fine-tuning of the window length aligns with the specific monitoring requirements of the user and the application. Lastly, we studied the sensitivity of the shift factor κ, experimenting with various values ranging from very small to larger, as shown in Fig. 8(c). A small shift factor is undesirable, as it corresponds to a low number of required instances that must be observed before declaring a change, making the model prone to detecting changes even with insignificant variations. Conversely, the results demonstrate stable and effective performance as the shift factor κ grows to moderate and large values, representing a higher number of required instances. These values ensure that significant changes are detected with greater confidence while avoiding excessive false positives triggered by transient or insignificant variations.

VII. Conclusion and Future Work

This paper has introduced RDSAD, an unsupervised, robust, and adaptive anomaly-based intrusion detection method for evolving complex CPS data streams. RDSAD integrates two innovative components: *Dynamic Deviation Recognition* (DDR) for seamlessly capturing underlying system dynamics and detecting attack patterns, and *Shift-aware Model Adaptation* for robust and adaptive model updates to accommodate changing system behaviours. Our experimental results across four benchmark datasets demonstrated that RDSAD achieves superior performance and an efficient runtime than state-of-the-art methods. This performance across various datasets illustrates the versatility and robustness of RDSAD, marking it as a promising solution for various dynamic applications. Nevertheless, we acknowledge that there is room for further improvement. RDSAD's performance depends on the assumption that the underlying distribution of normal behavior can be effectively modeled in the latent space. In highly stochastic or weakly structured systems, this assumption may not hold. Furthermore, we recognise potential challenges associated with directly deploying deep-learning-based methods such as RDSAD on low-power edge devices commonly used in CPS deployments. To practically address this challenge, future research directions could include lightweight model design and compression methods (e.g., pruning, quantization, or knowledge distillation) to streamline RDSAD for resource-constrained environments. Furthermore, while RDSAD's SMA mechanism effectively mitigates self-poisoning and catastrophic forgetting under standard anomaly scenarios, it currently lacks protection against adversarial attacks. In particular, adversaries may craft subtle input perturbations to manipulate deviation scores or trigger misleading shift detections, resulting in false negatives or unnecessary model updates. Addressing this vulnerability is a critical direction for future work. We plan to investigate robust adversarial training strategies tailored for unsupervised streaming contexts, including resilience to gradient-based attacks (e.g., FGSM, PGD) and consistency enforcement in latent space representations. Additionally, we plan to investigate more realistic mechanisms for simulating normality shifts, such as synthetically inducing gradual changes in the statistical properties of normal data (e.g., mean or variance drift) or introducing controlled latent space perturbations that mimic domain evolution over time. To support reproducibility and future research, we intend to publicly release the implementation of RDSAD, along with the evaluation scripts, upon publication. Finally, we aim to evaluate RDSAD on additional CPS domains such as power grids and autonomous systems to further assess its generalizability.

References

[1] H. Kayan, M. Nunes, O. Rana, P. Burnap, and C. Perera, "Cybersecurity of industrial cyber-physical systems: A review," *ACM Comput. Surv.*, vol. 54, no. 11s, pp. 1–35, 2022.

[2] J. Giraldo et al., "A survey of physics-based attack detection in cyber-physical systems," *ACM Comput. Surv.*, vol. 51, no. 4, 2018, Art. no. 76.

[3] A. Alsaedi, Z. Tari, R. Mahmud, N. Moustafa, A. Mahmood, and A. Anwar, "USMD: UnSupervised misbehaviour detection for multi-sensor data," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 724–739, Jan./Feb. 2023.

[4] D. U. Case, "Analysis of the cyber attack on the ukrainian power grid," *Electricity Inf. Sharing Anal. Center*, vol. 388, no. 1–29, 2016, Art. no. 3.

[5] The Florida water plant attack signals a new era of digital warfare. Accessed: Apr. 20, 2021. [Online]. Available: https://www.darktrace.com/en/blog/the-florida-water-plant-attack-signals-a-new-era-of-digital-warfare-its-time-to-fight-back/

[6] M. Ike, K. Phan, K. Sadoski, R. Valme, and W. Lee, "SCAPHY: Detecting modern ICS attacks by correlating behaviors in SCADA and physical," in *Proc. 2023 IEEE Symp. Secur. Privacy*, 2023, pp. 20–37.

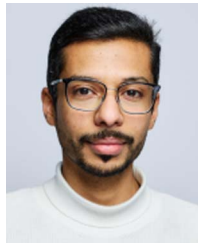
[7] Y. Luo, Y. Xiao, L. Cheng, G. Peng, and D. Yao, "Deep learning-based anomaly detection in cyber-physical systems: Progress and opportunities," *ACM Comput. Surv.*, vol. 54, no. 5, pp. 1–36, 2021.

[8] M. Fraccaro, S. K. Sønderby, U. Paquet, and O. Winther, "Sequential neural models with stochastic layers," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 2207–2215.

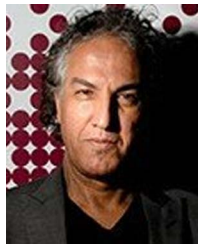
[9] J. Chung, K. Kastner, L. Dinh, K. Goel, A. C. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 2980–2988.

[10] Y. Song, J. Lu, H. Lu, and G. Zhang, "Learning data streams with changing distributions and temporal dependency," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 8, pp. 3952–3965, Aug. 2023.

[11] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
[12] D. Han et al., "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–13.
[13] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
[14] I. Higgins et al., "beta-VAE: Learning basic visual concepts with a constrained variational framework," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–12.
[15] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–19.
[16] P. Malhotra et al., "LSTM-based encoder-decoder for multi-sensor anomaly detection," 2016 *arXiv:1607.00148*.
[17] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Trans. Robot. Autom.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
[18] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, "Anomaly detection in streams with extreme value theory," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 1067–1075.
[19] M. Du, Z. Chen, C. Liu, R. Oak, and D. Song, "Lifelong anomaly detection through unlearning," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1283–1297.
[20] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3971–3988.
[21] F. Angiulli and F. Fassetti, "Detecting distance-based outliers in streams of data," in *Proc. 16th ACM Conf. Inf. Knowl. Manage.*, 2007, pp. 811–820.
[22] S. C. Tan, K. M. Ting, and T. F. Liu, "Fast anomaly detection for streaming data," in *Proc. 22nd Int. Joint Conf. Artif. Intell.*, 2011, pp. 1511–1516.
[23] S. Saurav et al., "Online anomaly detection with concept drift adaptation using recurrent neural networks," in *Proc. ACM India Joint Int. Conf. Data Sci. Manage. Data*, 2018, pp. 78–87.
[24] T. Pevnỳ, "Loda: Lightweight on-line detector of anomalies," *Mach. Learn.*, vol. 102, pp. 275–304, 2016.
[25] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Symp. Netw. Distrib. Syst. Secur.*, 2018, pp. 1–15.
[26] M. Delange et al., "A continual learning survey: Defying forgetting in classification tasks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3366–3385, Jul. 2022.
[27] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending TRANSCEND: Revisiting malware classification in the presence of concept drift," in *Proc. 2022 IEEE Symp. Secur. Privacy*, 2022, pp. 805–823.
[28] L. Li, J. Yan, Q. Wen, Y. Jin, and X. Yang, "Learning robust deep state space for unsupervised anomaly detection in contaminated time-series," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 6058–6072, Jun. 2023.
[29] M. D. Hoffman, D. M. Blei, C. Wang, and J. Paisley, "Stochastic variational inference," *J. Mach. Learn. Res.*, vol. 14, no. 5, pp. 1303–1347, 2013.
[30] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Representations*, 2014, pp. 1–14.
[31] M. Biloš, J. Sommer, S. S. Rangapuram, T. Januschowski, and S. Günnemann, "Neural Flows: Efficient alternative to neural ODEs," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 21325–21337.
[32] L. Tao, L. Feng, J. Yi, S.-J. Huang, and S. Chen, "Better safe than sorry: Preventing delusive adversaries with adversarial training," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 16209–16225.
[33] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv: 1803.01271*.
[34] A. Soule, K. Salamatian, and N. Taft, "Combining filtering and statistical methods for anomaly detection," in *Proc. 5th ACM SIGCOMM Conf. Internet Meas.*, 2005, pp. 31–31.
[35] D. Sejdinovic, B. Sriperumbudur, A. Gretton, and K. Fukumizu, "Equivalence of distance-based and RKHS-based statistics in hypothesis testing," *Ann. Statist.*, vol. 41, pp. 2263–2291, 2013.
[36] A. Alsaedi, N. Sohrabi, R. Mahmud, and Z. Tari, "RADAR: Reactive concept drift management with robust variational inference for evolving IoT data streams," in *Proc. IEEE 39th Int. Conf. Data Eng.*, 2023, pp. 1995–2007.
[37] T. Broderick, N. Boyd, A. Wibisono, A. C. Wilson, and M. I. Jordan, "Streaming variational Bayes," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2013, pp. 1727–1735.
[38] R. Kurle, B. Cseke, A. Klushyn, P. Van Der Smagt, and S. Günnemann, "Continual learning with Bayesian neural networks for non-stationary data," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–13.
[39] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4027–4035.
[40] X. Yang, E. Howley, and M. Schukat, "ADT: Time series anomaly detection for cyber-physical systems via deep reinforcement learning," *Comput. Secur.*, vol. 141, 2024, Art. no. 103825.
[41] Q. Xu, S. Ali, and T. Yue, "Digital twin-based anomaly detection with curriculum learning in cyber-physical systems," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 5, pp. 1–32, 2023.
[42] A. Hartl, F. Iglesias, and T. Zseby, "SDOstream: Low-density models for streaming outlier detection," in *Proc. Eur. Symp. Artif. Neural Netw. Comput. Intell. Mach. Learn.*, 2020, pp. 661–666.
[43] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.
[44] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.
[45] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. Int. Conf. Crit. Inf. Infrastructures Secur.*, Springer, 2016, pp. 88–99.
[46] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: A water distribution testbed for research in the design of secure cyber physical systems," in *Proc. 3rd Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2017, pp. 25–28.
[47] A. Vergara, S. Vembu, T. Ayhan, M. A. Ryan, M. L. Homer, and R. Huerta, "Chemical gas sensor drift compensation using classifier ensembles," *Sensors Actuators B: Chem.*, vol. 166, pp. 320–329, 2012.
[48] Scikit multiflow, "A machine learning package for streaming data in Python," Mar. 2022. [Online]. Available: https://scikit-multiflow.github.io/



Abdullah Alsaedi received the PhD degree from RMIT University, Melbourne, in 2024, with a specialization in cybersecurity, focusing on network systems. He is an assistant professor with the College of Computer Science and Engineering, Taibah University, Saudi Arabia. His research interests include cyber-physical systems (CPSs) cybersecurity, artificial intelligence, machine learning, and network security.



Zahir Tari is a full professor with Distributed Systems, RMIT University (Australia) and the research director with the RMIT Cyber Security Centre (CCSRI). His main expertise is in the areas of system's performance (e.g., P2P, cloud, edge, IoT) as well as system's security (e.g., SCADA, smart grid, cloud, IoT). He is an associated editor of the *ACM Computing Surveys* and was an associate editor of the *IEEE Transactions on Computers* (TC) and IEEE Transactions on Parallel.

Redowan Mahmud received the PhD degree from the School of Computing and Information Systems, University of Melbourne (Australia), in 2020. He is currently working as a lecturer (computing discipline) with the School of Electrical Engineering, Computing and Mathematical Sciences, Curtin University. His research interests include Internet of Things, fog and edge computing.